



Univerzitet u Novom Sadu, Prirodno-matematički fakultet

DEPARTMAN ZA MATEMATIKU I INFORMATIKU

Trg Dositeja Obradovića 4, 21000 Novi Sad

BVH konstrukcija ponderisana cenom primitiva za GPU ray tracing analitičkih scena

Master rad

Mentor: dr Marko Savić

Student: Nikola Pešić (57m/22)

Novi Sad, 2026

Predgovor

Ovaj master rad nadovezuje se na sadržaj predmeta Računarska grafika 2 na master akademskim studijama Prirodno-matematičkog fakulteta Univerziteta u Novom Sadu. Programska osnova razvijena u okviru predmeta obuhvatala je CPU implementaciju praćenja putanja, matematičke tipove, kameru, materijale i početne funkcije preseka sa sferom, kutijom, ravni i afino transformisanim primitivama. U okviru master rada na toj osnovi razvijeni su CUDA renderer, GPU zapis scene, standardna i ponderisana konstrukcija BVH strukture, srodne metode konstrukcije i eksperimentalna programska podrška.

Uz rad su priloženi izvorni kod i skup sirovih i izvedenih rezultata: <https://github.com/nikolapesic2802/master-rad>.

Zahvaljujem svom mentoru, dr Marku Saviću, na stručnom usmeravanju, savetima i podršci tokom izrade ovog master rada. Posebno sam mu zahvalan i na predavanjima tokom mojih studija, koja su ostavila poseban utisak na mene.

Zahvaljujem i svim profesorima Prirodno-matematičkog fakulteta Univerziteta u Novom Sadu na znanju koje su mi preneli i iskustvu koje sam stekao tokom studija.

Sažetak

Praćenje zraka (engl. ray tracing) za svaki segment svetlosne putanje traži najbliži presek sa scenom. Hijerarhija ograničavajućih zapremina (engl. Bounding Volume Hierarchy, BVH) ubrzava ovaj upit grupisanjem primitiva u stablo osno poravnatih ograničavajućih kutija (engl. Axis-Aligned Bounding Box, AABB). Heuristika zasnovana na površini (engl. Surface Area Heuristic, SAH) bira raspored prema površinama čvorova i procenjenom radu u listovima. Standardni objektni SAH procenjuje rad u listu samo brojem primitiva, iako testovi preseka sa različitim vrstama primitiva nemaju jednaku cenu na grafičkom procesoru.

Osnovna ideja rada je da cena sadržaja lista pri konstrukciji zavisi od kalibrisanih relativnih cena preseka sa sferom, kutijom, afinom sferom i afinom kutijom. Ispitana su četiri načina konstrukcije opisana u literaturi: objektni SAH, stabla razdvojena po tipu primitive, SBVH i lokalne rotacije, svaki u standardnoj i ponderisanoj varijanti. Metode su poređene vremenom CUDA kernela za ceo render na tri vrste opterećenja. Namenski konstruisana familija Layered Hero ispituje osetljivost i rast pravilne heterogene scene, Overlap proverava prostorne podele pri snažnom preklapanju AABB granica, a familije F1 - F4 proveravaju ponašanje na nasumičnim rasporedima i različitim sastavima primitiva.

Medijana promene vremena GPU kernela ponderisanog objektnog SAH prema standardnom objektnom SAH kroz pet skala familije Layered Hero iznosi 7,56%, pri čemu pozitivan znak označava kraće vreme. Ponderisanje skraćuje medijansko vreme i kod stabala po tipu, SBVH konstrukcije i lokalnih rotacija, uz učinak koji zavisi od porodice i scene.

Ključne reči: praćenje zraka, hijerarhija ograničavajućih zapremina, BVH, heuristika zasnovana na površini, SAH, GPU, CUDA, analitičke primitive, ponderisana konstrukcija, ponovljivost merenja

Sadržaj

1 Uvod.....	5
1.1 Motivacija.....	5
1.2 Cilj i istraživačka pitanja.....	5
1.3 Eksperimentalni pristup.....	6
1.4 Opseg istraživanja.....	6
1.5 Doprinos rada.....	6
1.6 Pristup proveru.....	7
1.7 Struktura rada.....	7
2 Teorijske osnove i pregled literature.....	8
2.1 Zrak i najbliži presek.....	8
2.1.1 Od jednačine renderovanja do niza zraka.....	8
2.2 Ograničavajuće kutije.....	9
2.2.1 Intervalni presek zraka i AABB granice.....	9
2.3 Hijerarhija ograničavajućih zapremina.....	9
2.3.1 GPU obilazak.....	10
2.3.2 Objektna konstrukcija od korena nadole.....	10
2.4 Standardni SAH.....	11
2.4.1 Pretpostavke heuristike.....	11
2.5 Postupci za poboljšanje BVH konstrukcije.....	11
2.5.1 Prostorne podele.....	11
2.5.2 Odvojena BVH stabla po vrsti primitive.....	12
2.5.3 Naknadno poboljšanje topologije.....	12
2.5.4 Raspodela zraka i vidljivost.....	12
2.5.5 Metode usmerene na brzinu izgradnje.....	13
2.5.6 Širi formati i drugačije granice.....	13
2.6 Pozicija rada.....	14
3 Renderer i podržane primitive.....	15
3.1 Poreklo koda i obim doprinosa.....	15
3.2 Provera ispravnosti renderera.....	16
3.3 Materijali i praćenje putanja.....	17
3.4 Uzorkovanje piksela.....	18
3.5 Zapis zraka i pogotka.....	19
3.6 Podržane primitive.....	20
3.6.1 Sfera.....	20
3.6.2 Osnovna poravnata kutija.....	20
3.6.3 Afino transformisana sfera.....	20
3.6.4 Afino transformisana kutija.....	21
3.6.5 Neograničena ravan.....	21
3.7 Arhitektura CPU i GPU izvršavanja.....	21
3.7.1 Linearni zapis primitiva.....	22
3.7.2 Kompaktan BVH zapis.....	22
3.7.3 Iterativni obilazak i stek.....	22
3.8 GPU izvršavanje.....	23
3.9 Determinizam i ponovljivost izvršavanja.....	23
4 Standardna objektna SAH konstrukcija.....	24
4.1 Ograničene primitive i konzervativne AABB granice.....	24
4.2 Objektna SAH konstrukcija.....	25
4.3 Zaustavljanje i determinističnost.....	26

4.3.1 Uticaj veličine lista na obični SAH.....	26
4.4 Kompaktan zapis stabla.....	27
4.5 Iterativni obilazak.....	28
5 SAH ponderisan cenom primitive.....	29
5.1 Cena sadržaja lista.....	29
5.2 Relativne težine operacija.....	30
5.3 Ponderisana ciljna funkcija.....	30
5.4 Uticaj na topologiju.....	31
5.5 Ponderisane varijante srodnih metoda.....	31
5.6 Modelovani geometrijski rad.....	31
5.7 Geometrijski primer promene podele.....	32
6 Eksperimentalna postavka.....	34
6.1 Eksperimentalne celine.....	34
6.2 Uvodno poređenje izvršnih puteva.....	34
6.3 Upareni redosled i provere ispravnosti.....	34
6.4 Glavno upareno merenje.....	35
6.5 Kalibracija cena operacija.....	36
6.6 Platforma.....	37
7 Rezultati.....	38
7.1 Linearni CPU i linearni GPU.....	38
7.2 Linearni GPU i GPU BVH.....	38
7.3 Osetljivost ponderisanog SAH na familiji Layered Hero.....	39
7.4 Scene završnog poređenja.....	42
7.4.1 Nasumične familije F1 - F4.....	42
7.4.2 Scena Overlap.....	42
7.5 Rezultati svih osam metoda.....	43
7.5.1 Layered Hero.....	44
7.5.2 Scena Overlap.....	45
7.5.3 Familije F1 - F4.....	45
7.6 Veličina, dubina i vreme konstrukcije.....	46
7.6.1 Layered Hero.....	48
7.6.2 Scena Overlap.....	49
7.6.3 Familije F1 - F4.....	49
8 Zaključak.....	50
8.1 Odgovori na istraživačka pitanja.....	50
8.2 Praktični značaj.....	51
8.3 Ograničenja i dalji rad.....	51
8.3.1 Validnost merenja.....	51
8.3.2 Opseg zaključaka.....	51
8.3.3 Pravci daljeg rada.....	52

Poglavlje 1

1 Uvod

1.1 Motivacija

Praćenje zraka (engl. *ray tracing*) gradi sliku iz velikog broja uzoraka. Svaki uzorak formira svetlosnu putanju od jednog ili više segmenata, a za svaki segment renderer traži najbliži presek sa scenom. Neposredna provera preseka svih objekata sa segmentom svetlosne putanje zato brzo postaje najskuplji deo računanja.

Hijerarhija ograničavajućih zapremina (engl. *Bounding Volume Hierarchy*, BVH) ubrzava pretragu grupisanjem primitiva u stablo. Svaki čvor čuva osno poravnatu ograničavajuću kutiju (engl. *Axis-Aligned Bounding Box*, AABB) svog podskupa. Kada zrak promaši kutiju, celo podstablo može da se odbaci bez ispitivanja njegovih primitiva. Dobro konstruisan BVH omogućava rad sa milionima objekata [1].

Kvalitet stabla zavisi od rasporeda primitiva po čvorovima. Heuristika zasnovana na površini (engl. *Surface Area Heuristic*, SAH) procenjuje cenu moguće podele iz površina dečjih kutija i rada koji sledi u njihovim listovima [2] [3]. Standardni objektni SAH procenjuje rad lista brojem primitiva koje list sadrži. U zapisu stabla one su predstavljene indeksima, koji se u nastavku nazivaju referencama. Takva procena je prirodna kada sve primitive koriste isti test preseka, kao u rendererima zasnovanim na trouglovima.

U heterogenoj analitičkoj sceni sa sferama, kutijama i njihovim afino transformisanim varijantama svaka vrsta primitive pokreće odgovarajuću GPU funkciju preseka. Isti broj primitiva zato ne mora da predstavlja isti rad. Dve podele sa sličnim prostornim granicama mogu imati različitu praktičnu cenu ako jedna od njih u često posećenim listovima ostavlja više skupih testova.

Izbor podele menja podskupove primitiva u deci, njihove AABB granice i sve odluke koje se zatim donose dublje u stablu. Razlika u proceni rada lista zato može da proizvede novu topologiju, drugačiji broj posećenih čvorova i drugačiji sastav listova. Geometrijski povoljnija topologija po modelu ipak ne mora uvek dati kraće vreme kernela. Memorijski pristupi, grananje niti i redosled instrukcija nisu u potpunosti opisani SAH izrazom. Učinak predložene izmene zato se proverava vremenom punog renderera, uz odvojeno praćenje modelovanog geometrijskog rada.

1.2 Cilj i istraživačka pitanja

Cilj rada je da ispita da li se BVH struktura može poboljšati kada se cena lista u SAH modelu računa iz kalibrisanih relativnih cena funkcija preseka. Ponderisana i standardna konstrukcija razmatraju iste moguće objektno podele i koriste isti format stabla, AABB granice i GPU postupak obilaska. Razlikuju se po cilju prema kojem se bira podela ili završetak u listu.

Evaluacija odgovara na tri pitanja:

1. Koliko se razlikuju cene GPU funkcija preseka za podržane vrste primitiva i kako se te razlike mogu ugraditi u cenu lista?

2. Da li ponderisani objektni SAH smanjuje vreme GPU kernela za ceo render na namenski konstruisanoj familiji Layered Hero i na nasumičnim scenama različitog sastava?
3. Kako se ponderisanje ponaša u stablima po tipu, prostornim podelama i lokalnim rotacijama, naročito na sceni projektovanoj za jako preklapanje?

Vreme GPU kernela je primarno merilo. Broj AABB testova, preseka sa primitivama i posećenih čvorova, kao i zauzetost steka, dopunjuju merenje i pomažu pri tumačenju razlika u vremenu. Vreme CPU konstrukcije prikazuje se odvojeno. Prenos scene i kopiranje rezultata nisu deo vremena GPU kernela.

1.3 Eksperimentalni pristup

Razvojne scene GI Test, City of Night i Signal Chamber različitog su obima i služe proveriti ispravnosti i izboru postavki. Uvodno poređenje linearnog renderera na CPU i GPU prikazuje doprinos paralelnog izvršavanja, a poređenje linearne GPU varijante i GPU varijante sa BVH strukturom prikazuje doprinos akceleratione strukture. Familija Layered Hero prati ponašanje ponderisane konstrukcije kroz pet veličina scene, sedam veličina lista i više vrednosti parametra ponderisanja. Dodatna scena sa 3.000 sfera i 7.000 dugih, tankih kutija namenski ispituje prostorne podele pri jakom preklapanju AABB granica. Familije F1 - F4 zatim proveravaju sve metode na po 100 nasumičnih rasporeda sa homogenim i heterogenim sastavom primitiva.

Poređenje obuhvata četiri metode: objektni SAH, stabla razdvojena po tipu primitive, prostorne podele SBVH i lokalne rotacije. Svaka je ispitana u standardnoj i ponderisanoj varijanti.

GPU studije koriste uparena merenja CUDA kernela za ceo render, dok se brojači geometrijskog rada prikupljaju odvojeno. U rezultatima se zato posebno prikazuju promena vremena i promena modelovanog geometrijskog rada.

1.4 Opseg istraživanja

Rad razmatra statične scene sa četiri vrste ograničenih analitičkih primitiva: sferama, osno poravnatim kutijama, afinim sferama i afinim kutijama. Neograničene ravni ostaju izvan BVH strukture. Sva poređenja koriste binarno stablo sa AABB granicama i isti softverski CUDA obilazak. Doprinos se zato može vezati za konstrukciju i njenu ciljnu funkciju, bez istovremene promene formata čvora ili hardverskog interfejsa za praćenje zraka. Kalibrisane težine važe za korišćeni GPU kod i uređaj, pa kalibraciju treba ponoviti na drugoj GPU arhitekturi ili posle promene funkcija preseka.

U opseg ne ulaze trouglovi, animirana geometrija, refitovanje stabla, široki ili kompresovani BVH formati i namenske hardverske jedinice za presek trouglova.

Razvojne scene služe izboru reprezentativnih postavki i proveriti ispravnosti. Brojčani zaključci zasnivaju se na pet skala Layered Hero, sceni Overlap sa 10.000 primitiva i na 100 deterministički generisanih rasporeda za svaku kombinaciju familije F1 - F4 i veličine 100, 1.000 ili 10.000 objekata.

1.5 Doprinos rada

Glavni doprinos rada je implementacija i eksperimentalno ispitivanje objektna BVH konstrukcije zasnovane na SAH modelu u kojoj cena lista zavisi od vrste primitive. Drugi doprinos je zajedničko poređenje četiri para standardnih i ponderisanih metoda: objektnog

SAH, stabala razdvojenih po tipu, SBVH i lokalnih rotacija. Uz izvorni kod čuvaju se sirova merenja, broj korena, čvorova, listova i referenci u izgrađenim strukturama, njihova dubina i veličina zapisa, kao i postavke korišćenih konfiguracija. Na osnovu tih podataka izvedene tvrdnje mogu se ponovo izračunati.

1.6 Pristup proveru

Ispravnost se proverava pre tumačenja performansi. Za iste pripremljene zrake objektni CPU upit poredi se sa linearnim GPU upitom, a svaka GPU BVH varijanta sa linearnim GPU rezultatom. Proveravaju se postojanje i rastojanje pogotka, materijal i normala, kao i poklapanje stabilnog rednog broja primitive u GPU putanjama. Posebne provere pokrivaju affine transformacije, konzervativnost granica i odsustvo prekoračenja GPU steka. Postupak je detaljnije opisan u odeljku 3.2.

Performanse se mere u neposrednim parovima nad istim zracima. Suprotni redosledi ABBA i BAAB smanjuju uticaj sporog pomeranja takta i temperature. Odnos kandidata i reference računa se unutar svakog od dva odvojena CUDA konteksta, a završni rezultat je medijana četiri odnosa dobijena iz blokova. Apsolutna vremena različitih konteksta se ne objedinjuju. Treći, instrumentisani kontekst prikuplja brojače, pa atomske operacije za dijagnostiku ne ulaze u primarno vreme. Rezultat se prikazuje kao procentualna promena, sa jasnim znakom i neposrednom referencom.

1.7 Struktura rada

Drugo poglavlje uvodi BVH, SAH i srodne konstrukcije. Treće opisuje renderer i primitive, četvrto standardni objektni SAH, a peto ponderisani cilj. Šesto daje kalibraciju i eksperimentalni postupak. Sedmo prikazuje i razmatra rezultate za Layered Hero, Overlap i familije F1 - F4, dok završno poglavlje odgovara na istraživačka pitanja.

2 Teorijske osnove i pregled literature

Ovo poglavlje uvodi geometrijske i algoritamske osnove potrebne za ostatak rada. Polazi od zraka i preseka sa AABB granicom, zatim opisuje BVH strukturu i SAH model, pa razmatra srodne konstrukcije i određuje mesto predloženog ponderisanog cilja u postojećoj literaturi. Detalji renderera, standardnog konstruktora i ponderisanog cilja dati su u trećem, četvrtom i petom poglavlju.

2.1 Zrak i najbliži presek

Zrak se zapisuje parametarski kao

$$r(t) = o + td, t \geq 0, \quad (2.1)$$

gde je o početak, a d pravac. Geometrijski upit se obično ograničava na interval $[t_{min}, t_{max}]$. Pozitivna donja granica sprečava ponovno pogađanje površine sa koje zrak polazi, a gornja granica se skraćuje čim se pronađe bliži pogodak.

Zapis najbližeg pogotka sadrži parametar t , stabilan redni broj primitive, položaj pogotka, geometrijsku normalu i materijal. Funkcija preseka računa ove podatke za svaki kandidat, a obilazak zadržava ceo zapis najbližeg kandidata. Redni broj je važan i za proveru implementacije: CPU i GPU moraju da izaberu isti objekat kada dve površine daju gotovo jednaku udaljenost.

Linearni postupak ispituje sve primitive i zadržava najbliži validan presek. Njegov rad raste linearno sa brojem objekata, pa u velikim scenama služi kao referenca za ispravnost. Praktični renderer koristi akceleracionu strukturu da bi većinu tih testova izbegao.

2.1.1 Od jednačine renderovanja do niza zraka

Jednačina renderovanja opisuje izlazno zračenje u tački kao zbir emitovanog zračenja i doprinosa koji stiže iz ostalih pravaca [4]. Numerički renderer taj integral procenjuje uzorcima. Zrak kamere pronalazi prvu površinu, materijal bira naredni pravac, a postupak se ponavlja dok se putanja ne završi ili ne dostigne dozvoljenu dubinu. Jedan uzorak slike zato obuhvata niz geometrijskih upita za najbliži pogodak. Materijali i način produžavanja putanje detaljnije su opisani u odeljku 3.3.

Sastav zraka se menja duž putanje. Primarni zraci susednih piksela često imaju slične polazne tačke i pravce. Posle odbijanja, prelamanja ili difuznog uzorkovanja postaju razućeniji. Površine koje su iz kamere zaklonjene mogu postati važne za sekundarne segmente. Zbog toga stablo koje je povoljno samo za prvi pogodak iz jedne kamere ne mora biti povoljno za puni renderer.

U ovom radu svaka CUDA nit prati jednu putanju, a BVH se poziva za svaki njen aktivni segment. Primarno vreme obuhvata ceo CUDA kernel i sve upite za najbliži pogodak koji nastanu duž putanje. Instrumentisani prolaz odvojeno broji segmente, AABB testove i testove primitiva, pa se promena topologije može povezati sa radom kroz celu putanju.

2.2 Ograničavajuće kutije

Ograničavajuća zapremina je jednostavna geometrija koja obuhvata objekat i čiji se presek računa jeftino. BVH u ovom radu koristi AABB, kutiju određenu donjim i gornjim uglom:

$$b_{min} = (x_{min}, y_{min}, z_{min}), b_{max} = (x_{max}, y_{max}, z_{max}). \quad (2.2)$$

Kutija mora da sadrži svaku tačku primitive. Premala granica može da sakrije stvarni pogodak i proizvede netačan rezultat, pa je konzervativnost uslov ispravnosti. Konzervativna kutija može da sadrži višak praznog prostora, što povećava verovatnoću nepotrebne posete.

Za dužine stranica e_x , e_y i e_z , površina AABB granice je

$$A(b) = 2(e_x e_y + e_x e_z + e_y e_z). \quad (2.3)$$

SAH model koristi odnos površina kao aproksimaciju verovatnoće da zrak koji je stigao do roditelja uđe i u određeno dete. Površine se tokom konstrukcije računaju neposredno iz AABB granica kandidata. Standardni model zato ne zavisi od određene kamere niti zahteva prethodno uzorkovanje zraka kroz scenu [5] [2].

2.2.1 Intervalni presek zraka i AABB granice

Presek sa kutijom dobija se presecanjem intervala duž tri ose. Za osu a računaju se parametri preseka sa njenim minimalnim i maksimalnim ravnima:

$$t_{0,a} = \frac{b_{min,a} - o_a}{d_a}, t_{1,a} = \frac{b_{max,a} - o_a}{d_a}. \quad (2.4)$$

Redosled ulaska i izlaska zavisi od znaka pravca. Zajednički interval kutije i aktivnog dela zraka dobija se izrazima

$$t_{ulaz} = \max(t_{min}, \min(t_{0,x}, t_{1,x}), \min(t_{0,y}, t_{1,y}), \min(t_{0,z}, t_{1,z})), \quad (2.5)$$

$$t_{izlaz} = \min(t_{max}, \max(t_{0,x}, t_{1,x}), \max(t_{0,y}, t_{1,y}), \max(t_{0,z}, t_{1,z})). \quad (2.6)$$

Pogodak postoji kada je $t_{ulaz} \leq t_{izlaz}$.

Nulte komponente pravca, potpisana nula, beskonačne recipročne vrednosti i položaj tačno na granici zahtevaju dosledna numerička pravila [6] [7].

2.3 Hijerarhija ograničavajućih zapremina

BVH je ukorenjeno stablo u kome unutrašnji čvor obuhvata geometriju svojih potomaka, a list čuva reference na dodeljene primitive. Referenca je indeks iz koga renderer pronalazi zapis odgovarajuće primitive. Korišćeni format je binaran, pa svaki unutrašnji čvor ima dvoje dece.

Obilazak počinje testom AABB granice korena, čiji promašaj odmah uklanja čitavo podstablo. U unutrašnjem čvoru testiraju se AABB granice oba deteta i porede njihove ulazne udaljenosti. Bliže pogođeno dete obilazi se prvo, dok udaljenije ostaje odloženo. Ako se u bližem detetu pronađe dovoljno blizak pogodak, skraćeni aktivni interval može da odbaci udaljeno dete pre ulaska u njegov sadržaj. U listu se ispituju sve primitive na koje upućuju njegove reference.

Ograničavajuće kutije treba tesno da obuhvate pripadajuću geometriju, sa što manje praznog prostora i međusobnog preklapanja, jer tesnije granice smanjuju broj nepotrebnih poseta. Sama veličina lista određuje drugi odnos: veći list smanjuje broj čvorova, ali

zahteva više pojedinačnih testova sa primitivama. Dublja topologija obično smanjuje broj takvih testova, a povećava broj AABB testova, memorijskih pristupa i operacija nad stekom. Zbog toga se veličina lista ispituje kao eksperimentalni parametar u odeljku 4.3, a njen izbor zavisi od scene i implementacije [1].

2.3.1 GPU obilazak

CUDA izvršava programske niti u grupama od po 32 koje se nazivaju engleskim terminom *warp*. Članovi jedne *warp* grupe dele tok instrukcija po modelu SIMT (engl. *Single Instruction, Multiple Threads*) [8]. Kada zraci biraju različite grane stabla ili funkcije preseka, izvršne putanje se serijalizuju za odgovarajuće podskupove. Na vreme utiču ukupan broj operacija i njihov raspored unutar grupe.

Hardver pri grananju privremeno aktivira samo programske niti koje prate istu putanju, a ostale čekaju da se izvršavanje ponovo spoji. Tako zrak koji promaši koren može rano da završi svoj rad, ali njegova *warp* grupa ostaje zauzeta dok susedni zraci prolaze kroz dublja ili jače preklopljena podstabla. Vreme grupe zato je bliže vremenu njenog najdužeg aktivnog puta nego proseku pojedinačnih puteva. Jednako smanjenje ukupnog broja testova može imati različit vremenski efekat u zavisnosti od toga da li su uklonjeni testovi raspoređeni kroz celu grupu ili koncentrisani u nekoliko njenih članova [9] [10].

GPU skriva čekanje na podatke prebacivanjem između više spremnih *warp* grupa. Broj grupa koje istovremeno mogu da borave na multiprocesoru ograničen je registrima, lokalnom memorijom i drugim resursima programa. Lokalni stek obilaska, odnosno niz odloženih čvorova koji vodi svaka CUDA nit, ulazi u taj račun. Pristupi globalnoj memoriji su povoljniji kada susedni članovi grupe traže bliske adrese. Koherentni zraci često posećuju iste ili susedne čvorove, dok divergentni zraci pristupaju udaljenim delovima niza. Format kompaktnog zapisa i postupak pakovanja zato su isti u svim poređenjima, iako ih SAH formula neposredno ne opisuje [8].

Na vreme utiču i red dece, trenutak kada je pronađen blizak pogodak i zauzetost steka [9]. Sama SAH vrednost zato ne objašnjava potpuno razlike u vremenu [10]. GPU vreme obuhvata celokupno izvršavanje, dok AABB brojači i brojači preseka sa primitivama izdvajaju geometrijski rad.

2.3.2 Objektne konstrukcija od korena nadole

Kod objektne podele svaka primitiva pripada tačno jednom detetu. Konstrukcija počinje svim referencama u korenu, bira osu i mesto podele, pa ponavlja postupak nad dva dobijena skupa. Kutije čvorova nastaju unijom AABB granica njihovih primitiva. Ovakav smer konstrukcije u literaturi se naziva *top-down*.

Jednostavni konstruktori dele uređeni skup po medijani centroida ili sredini najduže ose. Takva pravila brzo daju uravnoteženo stablo, ali ne procenjuju budući rad obilaska. SAH razmatra više kandidata i bira onaj sa najmanjom procenjenom cenom.

Velika ili izdužena primitiva može da proširi granicu jednog deteta duboko u prostor drugog. To je ograničenje objektne podele. Prostorne podele opisane u odeljku 2.5 nude drugačije rešenje.

2.4 Standardni SAH

Za roditelja P i kandidatsku podelu na skupove L i R , standardna cena može da se zapiše kao

$$C_{split} = C_{trav} + \frac{A(L)}{A(P)} N_L C_{isect} + \frac{A(R)}{A(P)} N_R C_{isect}, \quad (2.7)$$

gde su N_L i N_R brojevi referenci, C_{trav} cena obilaska jednog unutrašnjeg čvora, a C_{isect} prosečna cena preseka sa primitivom. Cena obilaska obuhvata čitanje unutrašnjeg čvora, proveru dečjih granica i izbor njihovog redosleda. Množenje površinom roditelja uklanja deljenja. Da bi poređenje ostalo ekvivalentno, istim faktorom množi se i cena lista:

$$\tilde{C}_{split} = A(P) C_{trav} + A(L) N_L C_{isect} + A(R) N_R C_{isect}. \quad (2.8)$$

Za list sa N_P referenci važe

$$C_{leaf} = N_P C_{isect}, \tilde{C}_{leaf} = A(P) N_P C_{isect}. \quad (2.9)$$

Konstruktor poredi najbolju legalnu vrednost $\tilde{C}_{split}$ sa $\tilde{C}_{leaf}$. Dodatna pravila zaustavljaju rekursiju pri degenerisanim centroidima ili prema zadatoj preferenciji veličine lista. Pravila izjednačenja moraju biti deterministička jer dva jednako ocenjena kandidata mogu da vode do različitih podela.

2.4.1 Pretpostavke heuristike

SAH svodi složen raspored zraka na lokalni odnos površina. U tom modelu se pretpostavlja približno ravnomerna raspodela pravaca, a svi testovi primitiva imaju jednu prosečnu cenu. Memorijska hijerarhija, divergencija izvršavanja, redosled dece i rano skraćivanje intervala ostaju izvan formule.

Uprkos tim idealizacijama, SAH je standardna osnova praktičnih BVH konstruktora. Praktične implementacije koriste prefiksne i sufiksne sume za potpunu evaluaciju kandidata ili grupišu centroide u konačan broj intervala radi skraćivanja vremena konstrukcije [3] [1].

Pretpostavka jedinstvene vrednosti C_{isect} problematična je u heterogenoj sceni, jer broj referenci ne razlikuje jeftine i skupe funkcije preseka. Opšti SAH model razdvaja implementaciono zavisnu cenu obilaska unutrašnjeg čvora C_{trav} od prosečne cene testa objekta C_{isect} [5] [2]. Ovaj rad prosečnu cenu testa dalje razlaže prema vrsti primitive i kalibriše je za četiri CUDA funkcije preseka: sferu, kutiju, afinu sferu i afinu kutiju.

2.5 Postupci za poboljšanje BVH konstrukcije

BVH se može menjati izborom prostornih kandidata, organizacijom primitiva, naknadnom izmenom topologije, procenom raspodele zraka ili vidljivosti, bržom izgradnjom i drugačijim formatom čvora ili granice [1]. U eksperimentalno poređenje ulaze prostorne podele, stabla po vrsti primitive i lokalne rotacije. Metode zasnovane na zracima i vidljivosti, brzi konstruktori i drugačiji formati razmatraju se radi pozicioniranja rada. Sve implementirane konstrukcije koriste isti binarni AABB zapis i GPU obilazak.

2.5.1 Prostorne podele

BVH sa prostornim podelama (engl. *Spatial Split BVH*, SBVH) može da postavi referencu iste primitive u oba deteta. AABB svake kopije ograničava se ravni podele sa odgovarajuće strane, čime se smanjuje preklapanje koje centroidna objektna podela ponekad ostavlja [11]. Izvorni algoritam razvijen je za trouglove, a implementirana varijanta

istu konstrukcionu ideju primenjuje na AABB granice analitičkih primitiva uz grupisane prostorne kandidate i ograničen broj duplikata. Ukupan broj referenci ograničen je na dvostruki početni broj, ista referenca može da se prostorno odseče najviše osam puta, a prostorni kandidati razmatraju se u čvorovima sa najmanje 16 referenci. Implementiran je i postupak pohlepnog uklanjanja duplikata (engl. *greedy unsplitting*). Posle izbora ravni on poredi duplirani zapis sa vraćanjem cele reference samo u levo ili samo u desno dete i uklanjanja duplikat kada jedna od tih mogućnosti smanji cilj bez pražnjenja deteta.

Duplirane reference i dalje opisuju jednu, nepodeljenu analitičku površinu i pozivaju isti test nad istom primitivom. Zrak koji pređe oba regiona može zato ispitati istu primitivu više puta. Prostorna podela je korisna kada smanjenje preklapanja i praznog prostora nadoknadi dodatne reference, testove i memoriju. Taj odnos je naročito važan za duge tanke objekte koji prolaze kroz velik deo scene.

2.5.2 Odvojena BVH stabla po vrsti primitive

Kod ove konstrukcije scena se čuva u više nezavisnih BVH stabala, po jednom za svaku prisutnu vrstu primitive. Svaki zrak proverava sve njihove korene, a za svaki pogođeni koren zasebno se obilazi odgovarajuće stablo. Scena sa dve vrste zato ima do dva, a scena sa četiri vrste do četiri nezavisna obilaska.

Prednost je što svaki list sadrži samo jednu vrstu primitive i ne mora da bira funkciju preseka za svaku referencu. Nedostatak je što jedan zrak može da pokrene više nezavisnih BVH obilazaka. Da li homogeni listovi nadoknađuju taj dodatni rad zavisi od scene.

2.5.3 Naknadno poboljšanje topologije

Lokalna rotacija, zasnovana na Kenslerovom postupku, menja grupisanje tri susedna podstabla bez ponovne izgradnje njihovih listova [12]. Ako je početni raspored $((A, B), C)$, ispituju se rasporedi $(A, (B, C))$ i $(B, (A, C))$, kao i dva simetrična kandidata kada je desno dete unutrašnji čvor. Na jednom čvoru zato se ispituju najviše četiri kandidata. Sadržaj listova, reference na primitive i broj čvorova ostaju isti. Menjaju se veze između posmatranih podstabala, njihove relativne dubine i AABB granice obnovljenih unutrašnjih čvorova. Za početni raspored i svaki kandidat računa se SAH cena lokalnog podstabla, a prihvata se raspored sa strogo manjom cenom.

Implementacija obrađuje stablo od listova ka korenu, pri čemu levo podstablo prethodi desnom, a kandidate svakog čvora ispituje fiksnim redosledom. Pri jednakim cenama zadržava se postojeći ili ranije izabrani raspored. Prolaz se ponavlja najviše četiri puta i završava ranije ako nije prihvaćena nijedna rotacija. Primitive i format čvorova ostaju isti, pa rezultat pokazuje koliko se dobija promenom grupisanja neposrednih suseda.

Šira pretraga malih podstabala može da pronađe poboljšanja koja jedna rotacija ne doseže. Karras i Aila paralelno optimizuju mala podstabla brzo izgrađene hijerarhije [13], dok ponovno umetanje traži povoljniji položaj celog podstabla [14]. Ovi širi postupci nisu implementirani, a eksperimentalno poređenje koristi prethodno opisane lokalne rotacije.

2.5.4 Raspodela zraka i vidljivost

Heuristika raspodele zraka (engl. *Ray Distribution Heuristics*, RDH) koristi reprezentativne zrake da proceni verovatnoću obilaska čvorova i taj signal kombinuje sa površinskom procenom [15]. Uzorak može da obuhvati zrake tekućeg, prethodnog ili više kadrova, pa kvalitet odluke zavisi od toga koliko dobro predstavlja kasnije upite. Autori beleže mali dobitak pri jednostavnom *ray castingu*, dok u svojim testovima punog praćenja zraka nisu izmerili korist.

Vinkler i saradnici uvode OSAH (engl. *Occlusion Surface Area Heuristic*), heuristiku zasnovanu na površini koja uključuje okluziju [16]. Vidljivi i nevidljivi trouglovi različito utiču na cilj, a signal vidljivosti kombinuje se sa površinskim odnosom i prostornim podelama. Metoda zato zahteva pripremnu procenu vidljivosti koja odgovara raspodeli zraka za koju se stablo gradi.

RDH menja procenu verovatnoće posete, dok OSAH uvodi signal vidljivosti i prostorne kandidate. Nijedna metoda ne menja cenu preseka prema vrsti primitive, pa bi se njihov signal mogao kombinovati sa ponderisanim sadržajem lista iz ovog rada. Takva kombinacija nije evaluirana i ostaje mogućnost za dalji razvoj i proveru ponderisanja primitiva.

2.5.5 Metode usmerene na brzinu izgradnje

Potpuna SAH evaluacija daje kvalitetnu lokalnu odluku, ali sortiranje i ispitivanje velikog broja kandidata može biti skupo. Linearni BVH (LBVH) najpre preslikava centroide u Mortonove kodove, uređuje ih i iz zajedničkih prefiksa kodova gradi hijerarhiju [17]. HLBVH grupiše primitive u prostorno koherentne klastere, brzo gradi njihove donje delove i zatim SAH postupkom povezuje gornje nivoe [18]. Ove metode su posebno važne kada se stablo mora često obnavljati.

Praktični konstruktori mogu da smanje broj SAH kandidata i bez prelaska na drugu porodicu metoda. PBRT grupiše centroide u 12 intervala jednake širine i ispituje 11 granica između njih [19]. Takav konačan skup može da izabere drugu podelu od potpune pretrage. Standardna i ponderisana konstrukcija u ovom radu ispituju svaku legalnu granicu između susednih centroida duž sve tri ose, pa u neposrednom poređenju polaze od istog skupa kandidata.

PLOC gradi hijerarhiju odozdo naviše. Aktivni klasteri čuvaju se u Mortonovom poretku, a svaki klaster traži najpovoljnijeg suseda u ograničenom lokalnom prozoru. Kao mera udaljenosti koristi se površina AABB granice koja obuhvata par, a uzajamno izabrani parovi spajaju se u paralelnim prolazima [20]. PLOC++ unapređuje pretragu suseda i organizaciju prolaza, čime poboljšava odnos kvaliteta stabla i trajanja izgradnje [21]. Ovi postupci ne ispituju sve legalne centroidne podele svakog roditelja kao potpuni objektni SAH. Prostor njihovih odluka određuju Mortonov poredak i lokalni prozor pretrage.

Paralelni brzi konstruktori uvode kompromis između trajanja izgradnje i kvaliteta stabla. Njihov glavni cilj je kraća izgradnja uz aproksimaciju ili drugačiji skup odluka, dok ovaj rad koristi potpunu evaluaciju svih legalnih centroidnih podela standardnog SAH modela unutar svakog čvora. Zbog toga brzi konstruktori nisu razmatrani kao zasebne metode.

2.5.6 Širi formati i drugačije granice

Postupci u ovom odeljku menjaju format čvora, način obilaska ili vrstu granice i pripadaju drugoj osi poređenja od ciljne funkcije ispitane u ovom radu. Promena formata utiče na veličinu čvora, raspored memorijskih pristupa i cenu jednog koraka obilaska, dok promena granice menja i test koji se izvršava nad svakim posećenim čvorom. Implementacija u ovom radu zadržava binarni AABB zapis i isti GPU postupak obilaska, čime se njen doprinos razdvaja od ovih pravaca razvoja akcelerationih struktura.

Široki BVH obrađuje više dece po čvoru, čime može da smanji broj nivoa stabla i bolje iskoristi vektorsko izvršavanje. Kompresovani široki zapis dodatno smanjuje memorijski promet tako što granice potomaka kodira u odnosu na zajednički roditeljski okvir [22]. Dobitak potiče iz drugačije topologije, zapisa podataka i koda obilaska. Nasuprot tome, obilazak bez steka čuva podatke potrebne za povratak u samom stablu ili u malom stanju

automata, pa uklanja eksplicitni stek uz dodatne veze i prelaze kroz hijerarhiju [23]. U oba slučaja menja se cena unutrašnjeg koraka koju SAH uzima kao osnovni parametar.

Orijentisana kutija može tesnije da obuhvati rotirane i izdužene objekte od AABB granice. Cena tog smanjenog praznog prostora je veći zapis i složeniji test koji zahteva transformaciju zraka ili računanje u lokalnom koordinatnom sistemu [24]. Granica k-DOP koristi više unapred izabranih pravaca ravni. Sa većim brojem pravaca postaje tesnija, ali raste broj intervala koje obilazak mora da učitati i ispita [25]. Struktura UBvh objedinjuje ograničavajuću zapreminu i geometriju lista u jednom zapisu, pa smanjuje broj odvojenih pokazivača i memorijskih pristupa karakterističnih za standardni BVH [26]. Te konstrukcije optimizuju zapis ili granicu.

2.6 Pozicija rada

Goldsmith i Salmon dopuštaju da čvorovi koriste sfere ili osno poravnate pravougaone prizme kao ograničavajuće zapremine i da se njihov očekivani broj testova vrednuje prema ceni preseka sa izabranom vrstom granice [5]. Te cene pripadaju testovima granica čvorova. U ovom radu granica ostaje AABB, a razlika u ceni preseka uvodi se među vrstama primitiva koje list sadrži.

MacDonald i Booth uvode razdvajanje cene prolaska kroz unutrašnji čvor i cene ispitivanja objekta, a površinski odnos koriste kao procenu uslovne verovatnoće posete [2]. To su članovi C_{trav} i C_{isect} uvedeni u odeljku 2.4.

Embree je prilagođen efikasnom CPU praćenju zraka sa širokim stablima i vektorizovanim jezgrima. Cena lista u tom okruženju može da prati broj vektorskih blokova trouglova koji se obrađuju zajedno [27]. Model je prilagođen SIMD organizaciji izvršavanja i ne kalibriše više analitičkih funkcija u mešovitom listu.

Predloženi metod zadržava binarni AABB zapis, objektne kandidate i isti GPU postupak obilaska. Standardni izraz $C_{isect}|S|$ za sadržaj lista zamenjuje zbirom kalibrisanih cena tipova:

$$\sum_{p \in S} c_{\tau(p)} \quad (2.10)$$

Ovde $\tau(p)$ označava vrstu primitive, a $c_{\tau(p)}$ relativnu cenu njene GPU funkcije preseka. Formalna definicija, kalibracija i način ugradnje u prefiksne i sufiksne sume dati su u petom i šestom poglavlju.

Rad prilagođava standardni SAH različitim CUDA funkcijama preseka, kalibriše njihove cene, deterministički gradi stabla i meri uparena GPU vremena u istom rendereru. Završni eksperimentalni skup poredi objektne i prostorne podele, stabla po tipu i lokalne rotacije uz istu osnovnu vrstu čvora.

3 Renderer i podržane primitive

Ovo poglavlje opisuje renderer u kojem su BVH konstrukcije implementirane i merene. Najpre razdvaja početni kursni kod od doprinosa rada, zatim uvodi putanje, uzorkovanje i podržane primitive, a završava CPU i GPU zapisom scene i načinom CUDA izvršavanja.

3.1 Poreklo koda i obim doprinosa

Polazna tačka bio je nastavni repozitorijum kursa Računarska grafika za školsku 2022/2023. godinu [28]. Iz tog koda potiču matematički tipovi, kamera, osnovni materijali, CPU renderer za praćenje putanja i matematički postupci preseka sfere, kutije i ravni, kao i generičko afino preslikavanje zraka i normale. CUDA realizacije razvijene su u ovom radu nad spakovanim zapisom tipa float, uz eksplicitne testove afine sfere i kutije, numeričku stabilizaciju i deterministički izbor najbližeg pogotka. Geometrija, raspored i kamera scene GI Test preneti su iz nastavnog materijala, uz prilagođavanje materijalnog zapisa GPU rendereru. City of Night polazi od kursne scene, ali je za ovaj rad ponovo oblikovana i znatno proširena. Signal Chamber je napravljena u okviru ovog rada.



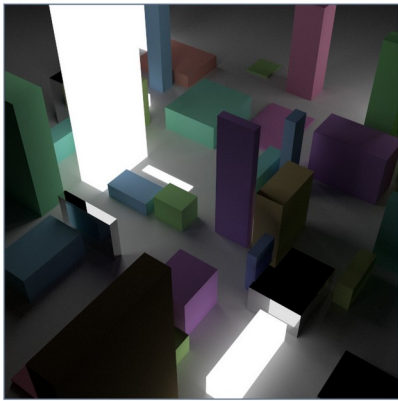
Slika 3.1. Razvojne scene GI Test, City of Night i Signal Chamber.

GI Test je mala zatvorena scena sa pet ravni: dva obojena bočna zida, pod, emisioni plafon i zadnji zid. U njoj se nalaze četiri sfere sa difuznim, ogledalskim i staklenim materijalom. Mali broj ograničenih primitiva čini pojedinačne efekte lako uočljivim, a refleksije, prelamanje i indirektno osvetljenje stvaraju dovoljno sekundarnih zraka za proveru dubine putanje, uzorkovanja i slaganja geometrijskih upita.

Pod i nizovi zgrada scene City of Night protežu se kroz dubinu kadra, dok emisioni fasadni paneli i veliko gornje svetlo daju složeniju raspodelu osvetljenja. Proširena scena sadrži 3.032 kutije i jednu neograničenu ravan. Razuđen urbani raspored služi kao srednje velika razvojna scena između malog GI Test primera i znatno gušće scene Signal Chamber.

Poreklo i razvoj scene City of Night

(a) Polazna scena iz nastavnog materijala · 641×640



(b) Proširena City of Night scena · 3840×2160



Slika 3.2. Razvoj scene City of Night: polazni render iz nastavnog repozitorijuma (levo) i prikaz scene proširene za ovaj rad (desno).

Signal Chamber je zatvorena reflektivna komora sa šest ravni i dva naspramna zida prekrivena pravilnim mrežama emisionih sfera. Svaki zid sadrži mrežu 101×151 , ukupno 30.502 ograničene primitive. Fiksna kamera gleda duž komore, pa ponovljena geometrija ispunjava dubok kadar bez promene kompozicije. Scena je uvedena kao razvojno opterećenje velikog broja objekata i jasno razdvaja linearnu pretragu od obilaska akceleratione strukture.

Neograničene ravni ispituju se izvan BVH strukture. Sadržaj stabla zato čine četiri sfere u sceni GI Test, 3.032 kutije u sceni City of Night i 30.502 sfere u sceni Signal Chamber.

U okviru master rada razvijeni su kompaktan GPU zapis scene, CUDA generisanje primarnih i sekundarnih zraka, linearni GPU upit, konstrukcija i obilazak više BVH varijanti, instrumentisani brojači i serije merenja. Dodate su i provere slaganja identiteta najbližeg pogotka između linearnog upita i upita koji koristi BVH strukturu. Renderi i glavna merenja obilaska dobijeni su GPU realizacijom. Uvodno poređenje uključuje i CPU renderer, a vreme konstrukcije BVH stabla meri se na procesoru.

3.2 Provera ispravnosti renderera

Tri razvojne scene koriste se za proveru implementacije pre merenja performansi. Najpre se za iste pripremljene zrake porede objektni CPU upit i linearni upit nad spakovanim GPU zapisom. U tom koraku proveravaju se postojanje i rastojanje pogotka, materijal i normala. Zatim se linearni GPU rezultat poredi sa GPU obilaskom svake BVH varijante. Između GPU putanja moraju da se poklope postojanje pogotka, stabilan redni broj primitive, materijal i normala, dok rastojanje mora da bude unutar zadate numeričke tolerancije.

Razvojne scene dopunjuju se namenskim zracima za numeričke granične slučajeve i afine transformacije. Posebne provere obuhvataju transformaciju zraka i normala, proveru da AABB granice ne odbace pogodak koji pronalazi linearni upit i proveru da zauzetost steka ostane unutar njegovog kapaciteta. Spakovani zapis tipa `float` detaljnije je opisan u odeljku 3.7.1, a izvođenje konzervativnih AABB granica iz tog zapisa u odeljku 4.1.

3.3 Materijali i praćenje putanja

Na mestu pogotka materijal određuje kako se svetlosna putanja nastavlja. Difuzna površina bira pravac u hemisferi iznad normale. Idealno ogledalo koristi reflektovani pravac

$$d_r = d - 2(d \cdot n)n, \quad (3.1)$$

staklo bira refleksiju ili prelamanje prema upadnom uglu i indeksu prelamanja. Totalna unutrašnja refleksija ostavlja samo reflektovani nastavak. Emitivna komponenta materijala neposredno dodaje svetlost rezultatu.

Ove mogućnosti u jednačini renderovanja opisuje funkcija rasipanja f_s [4]:

$$L_o(x, \omega_o) = L_e(x, \omega_o) + \int f_s(x, \omega_i, \omega_o) L_i(x, \omega_i) |n \cdot \omega_i| d\omega_i. \quad (3.2)$$

Izlazna radijansa L_o sastoji se od emitovane svetlosti L_e i doprinosa svetlosti pristigle iz drugih pravaca. Za proizvoljnu scenu integral se procenjuje uzorkovanjem. Ako se pravac ω_i bira iz gustine verovatnoće $p(\omega_i)$, doprinos jednog uzorka je

$$\frac{f_s(x, \omega_i, \omega_o) L_i(x, \omega_i) |n \cdot \omega_i|}{p(\omega_i)}. \quad (3.3)$$

CPU renderer u korišćenoj verziji procenjuje ovaj integral Monte Karlo uzorkovanjem svetlosnih putanja, a isti princip zadržan je u GPU realizaciji. Funkcija rasipanja, kosinusni faktor i gustina uzorkovanja objedinjeni su u izboru pravca i težini uzorka.

Praćenje putanja (engl. *path tracing*) procenjuje jednačinu renderovanja praćenjem slučajno uzorkovanih svetlosnih putanja [19]. U svakoj tački pogotka bira novi pravac i ponavlja upit najbližeg preseka, gradeći slučajnu svetlosnu putanju. Putanja počinje primarnim zrakom kamere. Difuzna odbijanja brzo rasipaju pravce, dok ogledala i staklo mogu da sačuvaju koherentniju grupu zraka i usmere je u drugi deo scene.

Uz putanju se vodi faktor prenosa (engl. *path throughput*), kojim se doprinos novog pogotka množi pre dodavanja rezultatu piksela. Najbliži pogodak određuje materijal, normalu i nastavak putanje, pa sve BVH topologije moraju da izaberu isti objekat i udaljenost za isti zrak.

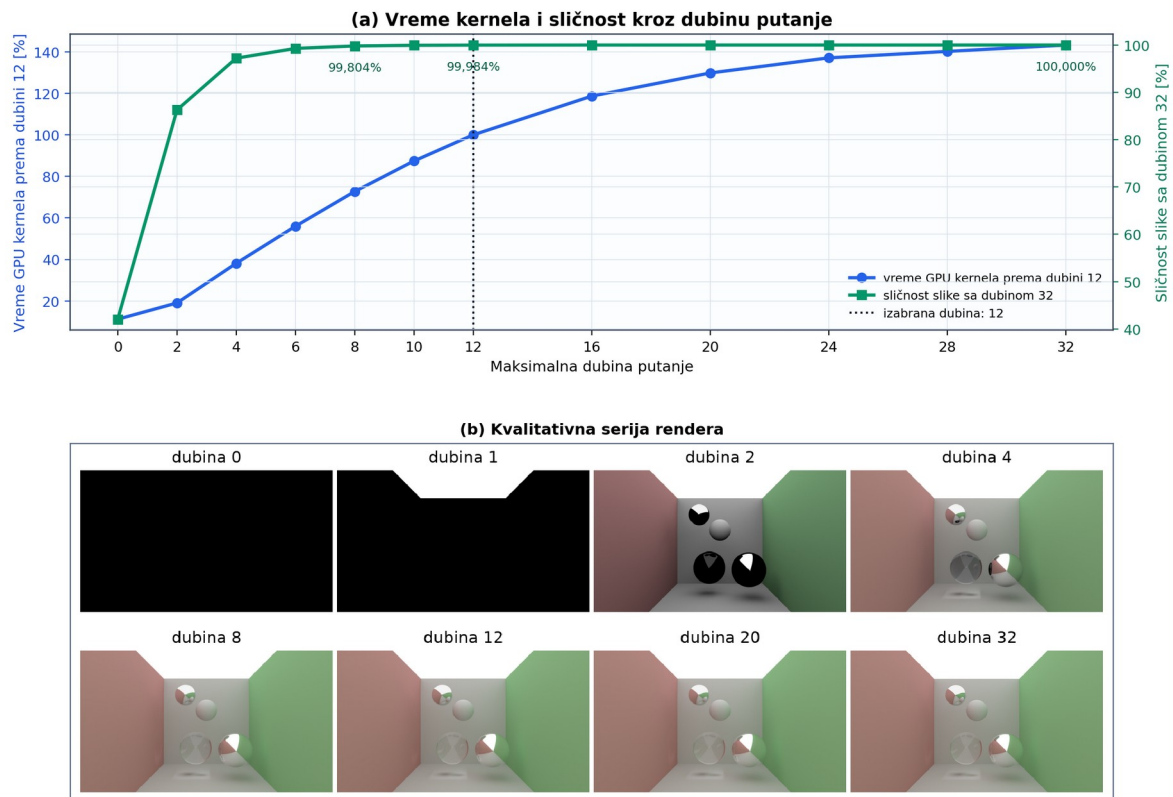
Najveća dubina određuje koliko dugo renderer može da nastavlja svetlosnu putanju kroz scenu. Mala maksimalna dubina prekida deo indirektnog osvetljenja, dok velika dubina povećava broj narednih geometrijskih upita. Taj odnos je ispitan na sceni GI Test pri postavkama koje se kasnije koriste u glavnim eksperimentima. Gornji grafikon na levoj osi prikazuje vreme GPU kernela za dubine 0, 2, 4, 6, 8, 10, 12, 16, 20, 24, 28 i 32 u odnosu na dubinu 12, pri 1920×1080 i jednom uzorku po pikselu. Svaki kandidat i dubina 12 izvršavaju se neposredno uparenim redosledom nad istim zracima. Prikazana vrednost je medijana četiri odnosa dobijena iz blokova, po dva iz svakog od dva CUDA konteksta. Referentna dubina 12 zato ima vrednost 100%, a uspravna linija označava postavku koja se koristi u glavnim eksperimentima.

Zelena kriva i desna osa na istom grafikonu kvantifikuju promenu slike na istoj mreži dubina. Svaki render ima 3840×2160 piksela i 512 uzoraka po pikselu, a scena, kamera, deterministička početna vrednost generatora pseudoslučajnih brojeva (engl. *seed*), BVH postavke i tonsko mapiranje (engl. *tone mapping*) jednaki su u celoj seriji. Sličnost osmootbitnih RGB slika posle tonskog mapiranja prema renderu dubine 32 definisana je kao

$$S_d = 100 \left(1 - \frac{1}{255N} \sum_{i=1}^N |I_d(i) - I_{32}(i)| \right), N = 3WH. \quad (3.4)$$

Vrednost $I_d(i)$ predstavlja osmootbitnu vrednost jednog RGB kanala posle tonskog mapiranja na spljoštenom indeksu i u renderu dubine d . Vrednost 100% označava potpuno poklapanje objavljenih piksela. Mera opisuje razliku prikaza posle tonskog mapiranja. Njena kriva se brzo približava referenci, dok vreme kernela nastavlja da raste.

Dubina putanje: vreme kernela i sličnost sa referentnim renderom



Slika 3.3. Odnos vremena GPU kernela prema referentnoj dubini 12 na levoj osi i sličnost slike posle tonskog mapiranja sa renderom dubine 32 na desnoj osi, na sceni GI Test. Gornji grafikon obuhvata ispitane dubine do 32, a donji niz prikazuje reprezentativne rendere dubina 0, 1, 2, 4, 8, 12, 20 i 32.

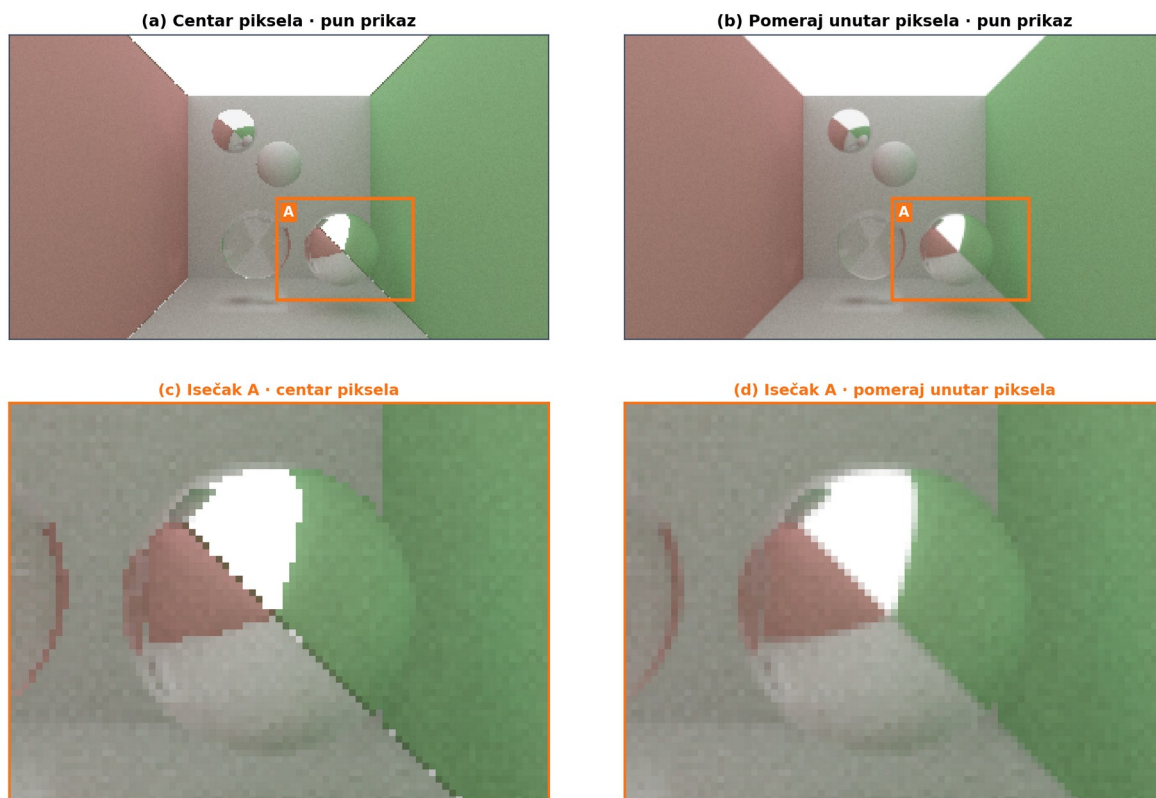
U ispitivanju dubina od 0 do 32, dubina 12 izabrana je jer daje 99,98% sličnosti sa renderom dubine 32 uz 30,20% kraće vreme.

3.4 Uzorkovanje piksela

Jedan piksel predstavlja malu oblast ravni slike. Zrak koji bi uvek prolazio kroz centar te oblasti ostavio bi izraženo stepenaste ivice. Renderer zato za svaki uzorak bira pseudoslučajnu tačku unutar piksela. Normalizovane koordinate te tačke mogu da se zapišu kao

$$u = \frac{x + \xi_x}{W}, v = \frac{y + \xi_y}{H}, \quad (3.5)$$

gde su (x, y) celobrojne koordinate piksela, W i H dimenzije slike, a ξ_x i ξ_y nezavisne pseudoslučajne vrednosti ravnomerno raspoređene tako da važi $0 \leq \xi_x < 1$ i $0 \leq \xi_y < 1$. Kamera preslikava (u, v) u pravac primarnog zraka. Usrednjavanje više ovako dobijenih uzoraka procenjuje integral preko površine piksela i ublažava nazubljenje (engl. *aliasing*).



Slika 3.4. Uticaj položaja primarnog zraka unutar piksela na sceni GI Test pri 320×180 i sa 512 uzoraka po pikselu. Uvećani isečak donje desne staklene sfere poredi zrake kroz centre piksela sa pseudoslučajno pomerenim zracima.

Pri uzorkovanju centara svi primarni zraci prolaze kroz isti položaj u odgovarajućem pikselu, pa zakrivljena silueta staklene sfere i kontrastna ivica u njenoj unutrašnjosti dobijaju stepenaste prelaze. Pseudoslučajni pomaci uzorkuju različite delove piksela i njihov prosek prikazuje delimičnu pokrivenost tih ivica.

U svim performansnim poređenjima identitet piksela, uzorka i dubine putanje određuje isti niz pseudoslučajnih vrednosti za sve poređene topologije.

3.5 Zapis zraka i pogotka

Uz početak i pravac, zrak čuva donju i gornju granicu t_{min} i t_{max} , između kojih se presek prihvata. Zapis najbližeg pogotka sadrži:

- oznaku da je presek pronađen
- parametar t najbližeg preseka
- stabilan redni broj pogođene primitive
- položaj pogotka
- geometrijsku normalu
- materijal i njegov indeks.

Stabilan redni broj razrešava slučaj dve skoro podudarne površine. Pakovanje za GPU preuređuje primitive radi kompaktnog pristupa memoriji, ali čuva vezu sa izvornim redosledom scene. Jednako udaljeni kandidati tako se biraju istim pravilom u objektnoj CPU referenci, linearnom GPU upitu i GPU BVH upitu.

Sekundarni zrak polazi neposredno sa površine. Aritmetika konačne preciznosti može da proizvede novi presek sa istom površinom pri veoma malom t , pa renderer koristi pozitivnu

donju granicu ili pažljivo pomenen početak. Ovo pravilo za izbegavanje samopresecanja (engl. *self-intersection*) zajedničko je svim varijantama geometrijskog upita.

3.6 Podržane primitive

Renderer podržava četiri vrste ograničenih analitičkih primitiva. Svaka ima konačnu AABB granicu izvedenu iz geometrije koja se šalje na GPU.

3.6.1 Sfera

Sfera sa centrom c i poluprečnikom r zadovoljava

$$\|x - c\|^2 = r^2. \quad (3.6)$$

Uvrštavanje zraka daje kvadratnu jednačinu u t . Funkcija preseka bira najmanji koren u aktivnom intervalu. Granica sfere dobija se neposredno kao $c \pm (r, r, r)$. Kratak zapis i mali broj operacija čine sferu prirodnom jediničnom osnovom pri kalibraciji cena.

Ako se uvede $m = o - c$, gde je o početak zraka, a c centar sfere, jednačina ima oblik $at^2 + 2bt + c = 0$, sa koeficijentima

$$a = d \cdot d, b = m \cdot d, c = m \cdot m - r^2. \quad (3.7)$$

Puna diskriminanta kvadratne jednačine jednaka je $(2b)^2 - 4ac = 4(b^2 - ac)$, pa njen znak određuje skraćeni izraz $D = b^2 - ac$ koji koristi implementacija. Negativna vrednost D označava promašaj, nula dodir u jednoj tački, a pozitivna vrednost dva realna korena. Prvo se proverava manji koren, a veći ostaje kandidat kada je prvi izvan aktivnog intervala. Ovaj drugi slučaj javlja se, na primer, kada zrak polazi unutar sfere. Tačka pogotka dobija se uvrštavanjem izabranog t u funkciju zraka $r(t) = o + td$ iz odeljka 2.1, a normala normalizovanjem vektora $r(t) - c$.

3.6.2 Oso poravnata kutija

Kutija je određena minimalnim i maksimalnim koordinatama, a presek koristi intervalni postupak iz prethodnog poglavlja. Geometrijska kutija mora da odredi lice, normalu i materijal, dok AABB granica čvora vraća samo podatak o pogotku i ulaznu udaljenost potrebnu za redosled obilaska. Pošto testovi vraćaju različite podatke, cene njihovih GPU funkcija mere se odvojeno.

Osa na kojoj je dostignut kraj zajedničkog intervala određuje lice geometrijske kutije. Znak pravca razlikuje minimalnu od maksimalne ravni, pa se normala dobija bez dodatnog ispitivanja svih šest lica. Kada zrak polazi unutar kutije, relevantan je izlazni kraj intervala.

3.6.3 Afino transformisana sfera

Afina transformacija spaja invertibilno linearno preslikavanje M , koje može da obuhvati rotaciju, skaliranje i smicanje, sa vektorom pomeranja p [19]. Oznake x_o i x_w predstavljaju koordinate iste tačke u objektnom i svetskom prostoru. Afina sfera nastaje preslikavanjem lokalne sfere u svetski prostor:

$$x_w = M x_o + p. \quad (3.8)$$

Presek se računa tako što se zrak prenese u objektni prostor,

$$o_o = M^{-1}(o_w - p), d_o = M^{-1}d_w. \quad (3.9)$$

Inverzna transformacija daje lokalni zrak čiji se pravac ne normalizuje. Zadržavanje njegove dužine čuva isti parametar t u objektnom i svetskom prostoru:

$$M^{-1}(o_w + t d_w - p) = o_o + t d_o. \quad (3.10)$$

Standardna formula sfere daje pogodak u objektnom prostoru. Normala se zatim prenosi inverzno transponovanim linearnim delom matrice i normalizuje. U odnosu na neposrednu sferu, ovaj postupak uvodi transformaciju zraka i normale [19].

Pravilo za normalu sledi iz zahteva da normala ostane upravna na svaku tangentu površine. Neuniformno skaliranje, na primer, različito menja dužine komponenti tangente. Kada bi se normala množila istom matricom M , njihov skalarni proizvod posle transformacije više ne bi morao da bude nula. Lokalni tangentni vektor v_o prelazi u $M v_o$, dok izbor $n_w = M^{-T} n_o$ daje $n_w^T M v_o = n_o^T v_o = 0$, a normalizacija daje jediničnu normalu. Inverzno transponovana matrica tako čuva ortogonalnost i pri neuniformnom skaliranju i smicanju.

Matrica M mora biti invertibilna, odnosno njena inverzna matrica mora da postoji. Pri pakovanju se proveravaju konačnost elemenata i numerička stabilnost inverzije. Transformacija koja ne prođe proveru saglasnosti preseka u CPU i GPU realizacijama odbacuje se pre konstrukcije BVH strukture.

Svetska AABB granica afine sfere računa se iz iste spakovane transformacije koju koristi GPU. Postupak njenog izvođenja dat je u četvrtom poglavlju, zajedno sa konzervativnim proširenjem granice.

3.6.4 Afino transformisana kutija

Afina kutija takođe prenosi zrak u lokalni prostor, gde se izvršava intervalni test. Njena funkcija preseka zato, pored intervalnog postupka, obuhvata i matične operacije. Rotirana ili smicana kutija ne ispunjava svoju svetsku AABB granicu, pa delovi granice izvan primitive mogu da izazovu nepotrebne posete. Kalibracija određuje relativnu cenu same funkcije preseka, dok labavija granica može da poveća broj kandidata koje struktura BVH dovodi do tog testa.

3.6.5 Neograničena ravan

Ravan nema konačnu AABB granicu i proverava se izvan stabla. Ona može da bude podloga, ogledalo ili drugi vizuelni element i time utiče na raspodelu sekundarnih zraka. Pošto ne učestvuje u podelama, njena kalibrisana cena pojavljuje se u modelu celog geometrijskog upita, ali ne u ciljnoj funkciji konstruktora.

3.7 Arhitektura CPU i GPU izvršavanja

Scena se najpre formira u CPU memoriji. Svaka primitiva ima vrstu, parametre, transformaciju, materijal i stabilan redni broj. Geometrijske formule za sferu, kutiju, ravan i afinu transformaciju potiču iz kursnog CPU koda. Za ovaj rad su preseki preneti u CUDA kod, sa četiri eksplicitne funkcije za ograničene primitive i zasebno funkcijom za ravan. Funkcije su prilagođene kompaktnom zapisu i pravilima numeričke saglasnosti. Objektni CPU zapis ostaje referenca za proveru, a iz njega nastaju linearni GPU nizovi i sve poređene BVH topologije.

Oba osnovna konstruktora polaze od istog ulaznog niza primitiva. Za svaku primitivu dobijaju istu pojedinačnu AABB granicu, ali tokom konstrukcije biraju kako će te primitive grupisati u čvorove. Zato mogu da proizvedu različite topologije, koje se zatim zapisuju u isti kompaktan format i obilaze istim GPU kodom.

3.7.1 Linearni zapis primitiva

GPU geometrija je organizovana na dva nivoa. Numerički parametri sfera, kutija, afinih sfera i afinih kutija čuvaju se u četiri odvojena kompaktna niza. BVH listovi zatim čuvaju zaseban niz kodiranih referenci, gde svaka referenca sadrži vrstu i indeks u odgovarajućem geometrijskom nizu. Linearni GPU upit prolazi direktno kroz geometrijske nizove, dok BVH upit do istih zapisa dolazi preko referenci iz posećenih listova.

Odvajanje sprečava da kratak zapis sfere nosi prazna polja potrebna afinom objektu. Homogeni list bira vrstu jednom, a mešoviti je čita iz svake reference, pa topologija može da promeni učestalost grananja. Zellmann i Lang porede jedno heterogeno BVH stablo sa odvojenim pokretanjem kernela za svaki tip primitive. Odvojeni prolazi u njihovom CUDA eksperimentu ubrzavaju samu fazu preseka, dok jedinstveno stablo izbegava više nezavisnih obilazaka [29].

Kompaktan zapis pomaže kada članovi jedne *warp* grupe pristupaju susednim primitivama, ali topologija ne garantuje takav obrazac. Zraci u istoj grupi mogu da stignu u različite listove i zatraže udaljene delove odgovarajućih nizova. Ovaj memorijski efekat ostaje deo neposrednog GPU vremena.

Pakovanje je determinističko. Jednaka scena i topologija daju isti redosled čvorova i referenci, što olakšava poređenje CPU i GPU izvršavanja.

3.7.2 Kompaktan BVH zapis

GPU dobija spljoštenu hijerarhiju umesto objektnje strukture konstruktora. Pokazivači, pojedinačni objekti i virtuelni pozivi napravili bi nepravilan obrazac pristupa i otežali prenos scene. Gusti nizovi sadrže AABB granice, metapodatke čvorova i reference primitiva. Unutrašnji čvor u metapodacima čuva indekse dece, dok list čuva početak i dužinu neprekidnog opsega referenci.

Isti format prihvata jedan koren standardnog objektnog stabla, više korena kod stabala po tipu i dodatne reference kod prostornih podela. GPU postupak obilaska zato ostaje isti za sve porodice konstruktora. Razlikuju se broj korena, topologija i inventar referenci, dok raspored polja i funkcije za presek ostaju zajednički.

Oznaka lista razlikuje homogeni sadržaj od mešovitog. Homogeni list bira jednu funkciju preseka pre petlje nad referencama. Mešoviti list čita vrstu iz svake reference. Ponderisanje može da promeni odnos ova dva slučaja i time utiče na grananje izvršnih niti, iako format samog lista ostaje isti.

3.7.3 Iterativni obilazak i stek

CUDA putanja obilazi stablo iterativno pomoću dva paralelna lokalna steka kapaciteta 32. Jedan čuva indekse odloženih čvorova, a drugi njihove već izračunate ulazne udaljenosti. Kada zrak pogodi obe dečje granice, udaljenije dete dodaje se prvo, a bliže drugo. Poslednje dodato dete prvo se obrađuje, pa pogodak u bližoj grani može da skрати t_{max} i odbaci udaljenu granu bez ulaska u njen sadržaj.

Kada čvor dođe na red, njegova AABB granica se ne ispituje ponovo. Dovoljno je poređenje sa trenutnim najbližim pogotkom. Instrumentisani prolaz beleži najveću zauzetost i svako prekoračenje, dok se pre merenja proverava i strukturna gornja granica. Rezervni linearni prolaz čuvao bi tačnost u slučaju prepunjenog steka, ali bi vremensko poređenje topologija izgubilo smisao. Završna merenja zato zahtevaju da broj prekoračenja bude nula.

3.8 GPU izvršavanje

Puni renderer koristi arhitekturu jednog velikog CUDA kernela (engl. *megakernel*) [30]. Taj kernel prati putanju od kamere do njenog prekida ili najveće dozvoljene dubine. Dvodimenzionalna mreža vezuje aktivnu programsku nit za jedan piksel i jedan uzorak. Nit izvodi pomake unutar piksela, formira primarni zrak i obrađuje sva naredna odbijanja. Uzorci istog piksela akumuliraju se kroz više pokretanja.

CUDA izvor se pri pokretanju prevodi pomoću biblioteke NVIDIA CUDA Runtime Compilation (NVRTC) u međukod Parallel Thread Execution (PTX). JCuda Driver API zatim učitava taj međukod i pokreće kernel [8].

Za proveru ispravnosti izdvojeni prvi pogodak koristi jednodimenzionalnu CUDA mrežu. Jedna aktivna nit obrađuje jedan unapred pripremljen zrak i poziva isti spakovani upit najbližeg pogotka kao puni renderer. Njegovo trajanje ne ulazi ni u jedan rezultat performansi.

3.9 Determinizam i ponovljivost izvršavanja

Ponovljivo poređenje zahteva stabilnost pre nego što se pokrene GPU. Scene se generišu iz eksplicitno zadate seed vrednosti, primitive dobijaju stabilne redne brojeve, a konstruktor koristi unapred određen redosled osa, kandidata i pravila izjednačenja. Pakovanje čvorova i referenci prati deterministički prefiksni obilazak. Jednaki ulazi zato daju istu scenu i istu topologiju.

Uz svako merenje beleže se podaci koji jednoznačno određuju korišćeni program, postavke protokola i kalibracije, kao i podaci o GPU uređaju. Vremenski i instrumentisani PTX čuvaju se odvojeno jer druga varijanta uključuje brojače. Pre objedinjavanja podataka proverava se da li svi zapisi potiču iz iste konfiguracije.

Determinističnost fiksira geometrijski i programski posao, dok takt, temperatura i raspored rada operativnog sistema i dalje menjaju trajanje. Upareni redosled opisan u šestom poglavlju ublažava tu preostalu vremensku varijaciju.

4 Standardna objektna SAH konstrukcija

Stablo se gradi u CPU memoriji pre slanja na GPU. Konstruktor polazi od spakovanih numeričkih vrednosti i AABB granica izvedenih iz istih float podataka koje će GPU koristiti za presek. U eksperimentima se koristi objektni SAH sa potpunom pretragom centroidnih podela. Standardna i ponderisana varijanta dele granice, skup kandidata, pravila zaustavljanja, GPU zapis i postupak obilaska, pa izbor topologije zavisi od izraza za cenu sadržaja lista.

4.1 Ograničene primitive i konzervativne AABB granice

Svaka ograničena primitiva dobija AABB granicu iz istog numeričkog zapisa koji se šalje na GPU. Afini objekti zavise od ovog pravila jer se njihove transformacije najpre zaokružuju na float zapis, a zatim se iz te predstave izvode granice. Konstruktor tako obuhvata geometriju koju će GPU zaista ispitivati.

GPU zapis neposredne sfere sadrži centar i poluprečnik. Te vrednosti se najpre zaokružuju na tip float, a krajevi AABB granice zatim se izvode iz dobijenog zapisa. Kod neposredne kutije upravo su minimalna i maksimalna tačka njeni osnovni GPU podaci, pa se ne izvode iz drugih geometrijskih parametara.

Za afinu primitivu inverzna linearna transformacija i svetski centar c zaokružuju se na isti float zapis. Inverzijom spakovane matrice dobija se direktna transformacija F , koja preslikava objektni prostor u svetski. Poluraspon transformisane jedinične sfere duž ose i jednak je euklidskoj normi odgovarajućeg reda:

$$e_i^{sfera} = \sqrt{F_{i0}^2 + F_{i1}^2 + F_{i2}^2}. \quad (4.1)$$

Za jediničnu kutiju $[-1, 1]^3$ poluraspon daje zbir apsolutnih vrednosti:

$$e_i^{kutija} = |F_{i0}| + |F_{i1}| + |F_{i2}|. \quad (4.2)$$

Početna granica obe primitive ima oblik $[c - e, c + e]$. I niz AABB granica koji se koristi pri proverama i kutije upisane u stablo nastaju istim funkcijama.

Svaki minimum granice primitive pomera se za dve susedne predstavljive float vrednosti naniže, a svaki maksimum za dva takva koraka naviše. Ovo malo proširenje štiti od gubitka stvarnog pogotka usled zaokruživanja. Kutije listova i unutrašnjih čvorova zatim se dobijaju unijom već proširenih granica, bez dodatnog pomeranja na višim nivoima.

Intervalni AABB test obrađuje svaku koordinatnu osu. Za nenultu komponentu pravca računa ulaznu i izlaznu vrednost, uređuje ih prema znaku i preseca sa aktivnim intervalom. Kod nulte komponente izbegava deljenje i proverava da li se početak zraka nalazi između dve ravni. Granice intervala blago se proširuju da bi rezultat ostao konzervativan uprkos zaokruživanju, u skladu sa poznatim robustnim pristupima [6] [7].

Pored podatka o pogotku, test vraća ulaznu udaljenost do kutije. Obilazak po toj vrednosti bira bliže dete i kasnije proverava da li je odložena grana ostala relevantna posle pronalaza novog pogotka.

Površina potrebna SAH modelu računa se nad tri dužine konzervativne granice. Prag od 10^{-12} za raspon centroida sprečava da numerički šum poslednjih bitova stvori besmisleni podelu između praktično jednakih centara.

4.2 Objektne SAH konstrukcije

Osnovni troškovni model i odnos rada u unutrašnjim čvorovima i listovima opisani su u odeljku 2.4. Ovde se zato navode samo odluke karakteristične za implementirani konstruktor. Oznaka C_T u nastavku predstavlja istu cenu unutrašnjeg koraka kao C_{trav} u opštem izrazu.

Konstruktor razmatra ose x , y i z . Primitive se sortiraju prema centru sopstvene AABB granice. Za jednu uređenu osu grade se prefiksne unije sleva i sufiksne unije zdesna. Kandidat između položaja $i-1$ i i tada dobija obe površine bez ponovnog obilaska svih elemenata.

Na svakoj legalnoj osi ocenjuje se svih $N-1$ nepraznih podela. To je puna pretraga diskretnih centroidnih kandidata, bez grupisanja u intervale. Isti prostor kandidata koristi i ponderisana varijanta. Nakon izbora, odgovarajući uređeni niz zamenjuje trenutni opseg radnog niza, a rekurzija se nastavlja nad njegova dva susedna dela.

Izraz „puna pretraga” označava iscrpnu evaluaciju konačnog skupa centroidnih podela. Ona uklanja aproksimaciju nastalu grupisanjem centroida u intervale, ali ne obuhvata proizvoljne prostorne ravni, odsecanje referenci ni sve binarne topologije. SBVH proširuje ovaj prostor kandidata dupliranjem i odsecanjem referenci.

Za uređeni niz $p_0, \dots, p_{N-1}$ kandidat na položaju i sadrži $p_0, \dots, p_{i-1}$ levo i preostale reference desno. Prefiksna granica B_i^L predstavlja uniju prvih i AABB granica, dok sufiksna B_i^R obuhvata ostatak. Njegova promenljiva cena je

$$A(B_i^L)i + A(B_i^R)(N-i). \quad (4.3)$$

Član $A(P)C_T$ jednak je za sve kandidate istog roditelja, ali se zadržava u punoj ceni zbog poređenja sa listom. Najveći dodatni trošak jednog čvora predstavlja uređivanje primitiva prema centroidu AABB granice. Prefiksni i sufiksni prolaz zatim linearno pripremaju granice potrebne za ocenu svih položaja podele. Posle podele isti postupak se ponavlja nad dva manja podskupa.

Izabrani minimum je lokalna odluka. Posle podele konstruktor nezavisno obrađuje dva deteta, pa pohlepna konstrukcija od korena nadole ostaje aproksimacija globalno najbolje binarne topologije [1].

Zamena jednog od dva slično ocenjena kandidata menja skupove primitiva u oba deteta i kandidate dostupne u svim njihovim potomcima. Mala promena ciljne cene zato može da proizvede veliku promenu topologije kroz više nivoa. Dok lokalni minimumi ostanu isti, isti interval vrednosti težina daje potpuno jednaku topologiju.

Primitiva se u celini dodeljuje jednom detetu, pa broj referenci ostaje jednak broju ograničenih objekata. Njena kutija se ne odseca ravni podele, zbog čega se AABB granice dece mogu preklapati i kada su njihovi centroidi razdvojeni. SBVH pokušava da smanji takvo preklapanje dupliranjem reference i ograničavanjem njenih dečjih granica ravni prostorne podele.

4.3 Zaustavljanje i determinističnost

Preferirana veličina lista određuje kada je podela obavezna, a kada zavisi od cene SAH modela. Važe sledeća pravila:

- jedna primitiva uvek završava u listu
- skup bez legalne centroidne podele završava u listu
- iznad preferirane veličine primenjuje se najbolja legalna podela
- na ili ispod preferirane veličine podela se prihvata kada je strogo jeftinija od lista.

Preferencija zato ima dve uloge. Iznad njenog praga sprečava veoma velike listove kada postoji makar jedna geometrijski legalna podela. Na ili ispod praga grananje se prekida čim dodatni unutrašnji korak više ne nadoknađuje očekivano smanjenje rada u listu. Ovakvo pravilo se razlikuje od fiksne veličine lista, kod koje bi svaki skup bio deljen do propisanog broja referenci bez obzira na površine.

Cena lista množi površinu roditelja brojem njegovih referenci, dok cena podele sabira unutrašnji član i površinski ponderisan sadržaj oba deteta. Stroga nejednakost čuva list kada su vrednosti jednake i time daje jednoznačnu graničnu odluku. Kod čvora iznad preferencije ista jednakost vodi u podelu, jer ograničenje veličine tada ima prednost nad lokalnim prekidom prema SAH modelu.

Stvarna popunjenost zato može da pređe preferiranu veličinu kod degenerisanih centara, a može biti i manja kada SAH opravda dalje grananje.

Deterministička topologija počinje stabilnim poretom. Kada dve primitive imaju isti centroid, prednost određuje njihov izvorni redni broj. Kandidati se obilaze fiksnim redosledom osa i položaja, a postojeći minimum se menja tek pri strogo manjoj ceni. Jednaka cena zato uvek zadržava ranije razmatrani kandidat. Ponderisana konstrukcija nasleđuje ovo pravilo, pa pri jediničnim cenama daje istu topologiju kao standardna konstrukcija.

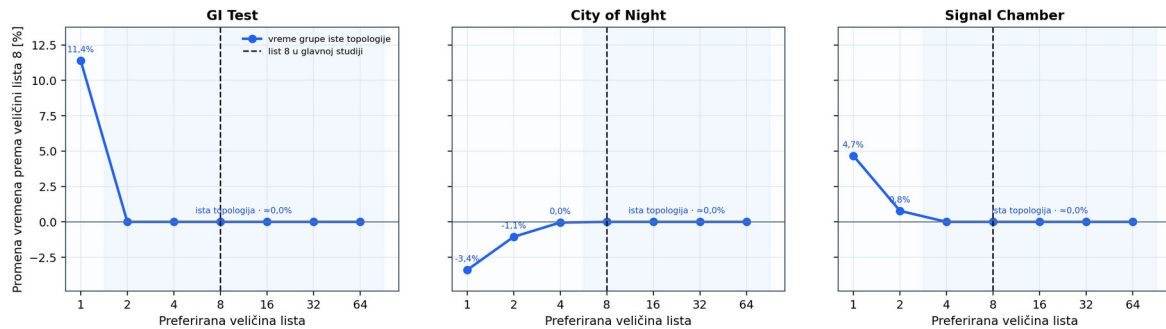
Veličina lista ostaje eksperimentalni faktor jer menja odnos između AABB rada i rada u listovima. Najpovoljnija vrednost zavisi od scene, funkcija preseka, memorijskog rasporeda i GPU obilaska [31].

4.3.1 Uticaj veličine lista na obični SAH

Gornja granica od osam primitiva po listu korišćena je u GPU studiji Aile i Lainea i u eksperimentima OSAH metode, dok PBRT podrazumevano koristi četiri primitive [9] [16] [19]. Reč je o postavkama konkretnih implementacija, bez tvrdnje da su teorijski optimalne za svaku scenu.

Obični objektni SAH meri se na tri razvojne scene sa preferencijama $\{1, 2, 4, 8, 16, 32, 64\}$, pri 1920×1080 , sa jednim uzorkom po pikselu i najvećom dubinom putanje 12. Scene su predstavljene u trećem poglavlju, a sve ostale postavke ostaju jednake. Svaka veličina neposredno se poredi sa postavkom veličine lista 8 u pet nezavisnih vremenskih konteksta. Jedan kontekst daje blok ABBA i blok BAAB, a konačni odnos je medijana deset procena dobijenih iz blokova. Detaljan upareni postupak opisan je u odeljku 6.3. Veličine koje daju istu topologiju prikazane su istom vrednošću. Grupa kojoj pripada veličina 8 postavljena je na 0%, pa negativne vrednosti označavaju kraće, a pozitivne duže vreme.

Obični objektni SAH: uticaj preferirane veličine lista
pet konteksta x dva bloka · ista topologija je objedinjena



Slika 4.1. Promena vremena običnog objektnog SAH prema grupi koja sadrži veličinu lista 8 na svakoj razvojnoj sceni. Veličine lista koje daju isto stablo objedinjene su istom vremenskom procenom. Negativna vrednost označava kraće vreme.

Najpovoljniju topologiju na sceni GI Test daju veličine lista 2, 4, 8, 16, 32 i 64. Najpovoljniju topologiju na sceni City of Night daje veličina lista 1. Najpovoljniju topologiju na sceni Signal Chamber daju veličine lista 4, 8, 16, 32 i 64. Različiti minimumi potvrđuju da jedna preferirana veličina nije najbrža na svakoj sceni.

Za glavna poređenja zadržava se veličina 8, koja je korišćena u referenciranim radovima [9] [16] i ostavlja SAH modelu mogućnost da ranije završi grananje kada dalja podela nije povoljna. To je zajednička kontrolisana postavka, bez pretpostavke da je najbrža na svakoj sceni.

Isti parametar se u sedmom poglavlju ponovo menja na familiji Layered Hero, gde se posmatra i njegov odnos sa ponderisanjem.

4.4 Kompaktan zapis stabla

Po završetku se stablo prefiksno spljoštava u tri niza:

1. šest vrednosti tipa float za AABB granicu svakog čvora
2. četiri celobrojne vrednosti metapodataka po čvoru
3. kodirane reference primitiva u redosledu listova.

Unutrašnji čvor u metapodacima čuva indekse levog i desnog deteta. List čuva oznaku sadržaja, početak neprekidnog opsega i broj referenci. Oznaka razlikuje mešoviti sadržaj od homogenog lista sfera, kutija, afinih sfera ili afinih kutija. Homogeni list bira funkciju preseka jednom pre petlje. Mešoviti list čita vrstu svake reference, nakon čega oba slučaja pozivaju iste funkcije i isto pravilo izbora najbližeg pogotka.

AABB niz i metapodaci su razdvojeni. Promašaj granice zato ne zahteva čitanje oznake lista, opsega referenci ni indeksa dece. Metapodaci se čitaju tek posle potvrđenog ulaska u čvor. Prefiksni raspored daje svakom podstablu neprekidan opseg čvorova. Prvo dete se nalazi neposredno iza roditelja, dok indeks drugog zavisi od veličine prvog podstabla i ostaje eksplicitno zapisan.

Objektni BVH ima jedan koren i jednu referencu po ograničenoj primitivi. Stabla po tipu imaju po jedan koren za svaku prisutnu vrstu, dok prostorne podele mogu povećati ukupan broj referenci. Sve poređene konstrukcije koriste isti format, pa se njihov memorijski zapis razlikuje prema broju korena, čvorova i referenci.

4.5 Iterativni obilazak

GPU obilazi spljošteno stablo pomoću dva lokalna steka kapaciteta 32. Prvi čuva indeks čvora, a drugi već izračunatu ulaznu udaljenost. AABB granica svakog korena proverava se na početku. Pogođeni koreni ulaze na stek uređeni po ulaznoj udaljenosti, pa se najbliži obrađuje prvi. Glavne konstrukcije imaju samo jedan koren, dok je ovaj početni korak važan za konstrukciju sa odvojenim stablima po tipu.

Sačuvana ulazna vrednost uklanja potrebu za ponovnim AABB testom kada čvor dođe na red. Najpre se poredi sa trenutnim najbližim pogotkom. Čvor čiji je ulaz postao predalek odbacuje se pre čitanja metapodataka.

Kod unutrašnjeg čvora testiraju se granice oba deteta. Stek najpre vraća poslednju dodatu stavku (engl. *last in, first out*, LIFO), pa se udaljenije dete dodaje prvo, a bliže drugo. Bliže se zato obrađuje prvo. Pogodak iz tog podstabla skraćuje aktivni interval i često uklanja ranije odloženo dete.

Obrada bližeg deteta pre udaljenijeg utiče na količinu rada, ali ne na matematički rezultat najbližeg preseka. Oba pogođena deteta ostaju kandidati dok njihova ulazna udaljenost nije iza trenutnog pogotka. Prednost bližeg samo povećava verovatnoću da će se t_{max} rano skratiti. Kada se dečje kutije preklapaju, ulazna udaljenost nije garancija da dete sadrži i bližu primitivu. Udaljena grana zato ostaje na steku sve dok novopronađeni pogodak zaista ne omogući njeno odbacivanje.

Zauzetost steka i trenutna dubina opisuju različite osobine obilaska. Putanja koja na svakom nivou pogađa oba deteta može da dodaje po jednu udaljenu granu, dok putanja kroz kutije bez preklapanja često ima svega nekoliko odloženih čvorova. Zabeleženi maksimum zato dopunjuje strukturnu dubinu i opisuje praktično opterećenje GPU steka.

Pri poređenju sa već pronađenim pogotkom obilazak koristi prethodno opisano konzervativno proširenje gornje granice intervala. Time kandidati na praktično istoj udaljenosti ostaju dostupni, a potpuno jednake pogotke razrešava manji stabilan redni broj primitive.

U listu se ispituje svaka njegova referenca. Poseban instrumentisani prolaz beleži AABB testove, posećene čvorove i listove, homogenost listova, najveću zauzetost steka, prekoračenja kapaciteta i testove po vrsti primitive.

Za jedan koren i opisani način obilaska postoji korisna kontrolna jednakost

$$N_{AABB} = 1 + 2 N_{unutrašnji}, \quad (4.4)$$

jer koren dodaje jedan test, a svaki posećeni unutrašnji čvor testove dve dečje granice. Kod šume se jedinica zamenjuje brojem ispitanih korena. Čvor pri vađenju sa steka koristi sačuvanu ulaznu udaljenost i zato ne testira sopstvenu granicu drugi put. Odstupanje bi ukazalo na drugačiji izvršni put ili grešku u instrumentaciji.

Prepunjen stek aktivira rezervni linearni prolaz kroz ograničene primitive. Taj put čuva geometrijsku ispravnost, ali bi sakrio kvalitet stabla i znatno promenio vreme. Završna merenja zato zahtevaju da nema prekoračenja, da izmerena zauzetost bude manja od 32 i da strukturna dubina ne može da popuni stek.

5 SAH ponderisan cenom primitive

Standardni objektni SAH svakoj primitivi u listu dodeljuje jediničnu cenu. Ponderisana varijanta umesto nje koristi relativnu cenu odgovarajuće GPU funkcije preseka. Promena ulazi u prefiksne i sufiksne sume tokom izbora podele, dok su geometrijski kandidati, pravila zaustavljanja, zapis čvorova i GPU obilazak preuzeti iz prethodnog poglavlja.

5.1 Cena sadržaja lista

Dva lista mogu da imaju isti broj referenci, a da zahtevaju veoma različit geometrijski rad. Presek sa sferom koristi kratak zapis i rešava kvadratnu jednačinu. Afina kutija zahteva transformaciju zraka, intervalni test u lokalnom prostoru i povratak normale u svetski sistem. Njen zapis je znatno veći. Brojanje referenci tu razliku ne registruje.

Ako list S sadrži n_k referenci vrste k , njegova opšta cena može da se napiše na dva jednaka načina:

$$C(S) = \sum_{i \in S} c_{\tau(i)} = \sum_k n_k c_k. \quad (5.1)$$

Oznaka $\tau(i)$ daje vrstu primitive na koju upućuje referenca i , a $c_{\tau(i)}$ relativnu cenu njene funkcije preseka. Ekvivalentan zapis preko n_k grupiše reference po vrsti. Standardni objektni SAH dobija se izborom $c_k=1$ za svaku vrstu. Taj izbor je prikladan kada sve reference pokreću istu funkciju nad istim zapisom.

Kod heterogenih analitičkih primitiva broj n_k poznat je već tokom konstrukcije. Ponderisana cena za list sabira unapred kalibrisane cene svih testova koje bi njegova poseta pokrenula. Isti zbir koristi se pri oceni leve i desne strane svake kandidatske podele. Geometrijski odnos površina i dalje procenjuje verovatnoću posete, a ponderisanje preciznije opisuje rad posle te posete.

SAH poredi relativne vrednosti. Množenje svih cena primitiva i C_T istom pozitivnom konstantom ne menja nijednu odluku. Promena cena primitiva uz fiksni C_T menja dve stvari. Različit odnos među vrstama može da izabere drugi prostorni kandidat, a veći zbir prema ceni unutrašnjeg koraka može da opravda dodatni nivo stabla. U homogenim scenama težina menja samo odluku između lista i podele, dok u heterogenim scenama može da promeni i izbor prostornog kandidata.

Ponderisanje u ovom radu koristi ovaj oblik za četiri analitičke CUDA funkcije. Relativne cene određuju se merenjem implementiranih funkcija i zatim ulaze u konstrukciju. Standardna i ponderisana varijanta polaze od istog redosleda primitiva i razmatraju iste prostorne kandidate. Odnos prema postojećim troškovnim modelima opisan je u odeljku 2.6.

5.2 Relativne težine operacija

Kalibracija obuhvata sedam operacija: testove preseka sa četiri vrste ograničenih primitiva, test preseka sa ravni, AABB test i obradu unutrašnjeg čvora. Vrednosti se normalizuju prema sferi, čija je težina jedan.

Težine sfere, kutije, affine sfere i affine kutije ulaze u cenu budućeg lista. Iste četiri vrednosti množe odgovarajuće ostvarene brojače pri računanju modelovanog geometrijskog rada. Ravan se nalazi izvan BVH strukture. Njena cena se dodaje modelu celog upita kada scena sadrži ravni, ali ne može da utiče na podelu.

Težina AABB testa pripada ostvarenim brojačima. Cena unutrašnjeg koraka C_T koristi se pri konstrukciji i obuhvata proveru dve dečje granice i izbor redosleda. Dodavanje i punog unutrašnjeg koraka i njegovih već zabeleženih AABB testova u model ostvarenog rada dvaput bi obračunalo isti deo izvršavanja. Zato model koristi pojedinačne AABB brojače, a C_T ostaje u SAH formulama.

Težine su relativni troškovi korišćenih GPU funkcija i važe za konkretan uređaj, prevodilac i raspored podataka.

Promena formata zapisa može da uvede dodatna učitavanja, druga verzija prevodioca da promeni raspored instrukcija, a druga GPU arhitektura odnos propusnosti aritmetike i memorije. Čak i na istom uređaju odnos pogodaka i promašaja u kalibracionom skupu utiče na aktivirane grane funkcije preseka. Zbog toga se pri kalibraciji koriste iste funkcije preseka kao u merenjima i više profila zraka, a dobijeni vektor treba ponovo izmeriti kada se promeni izvorni kod ili platforma.

5.3 Ponderisana ciljna funkcija

Neka je c_i kalibrisana relativna cena primitive i , uz $c_s=1$ za sferu. Parametar λ kontroliše jačinu signala:

$$q_i(\lambda) = 1 + \lambda(c_i - 1). \quad (5.2)$$

Pri $\lambda=0$ sve težine su jednake jedinici. Vrednost $\lambda=1$ neposredno koristi kalibrisane odnose. Interval između nule i jedinice ublažava razlike, a veće vrednosti ih namerno pojačavaju radi analize osetljivosti. Težina sfere ostaje jednaka jedinici duž cele ose λ .

Implementacija zahteva da kalibrisane cene c_i , a time i izvedene težine $q_i(\lambda)$, budu konačne i strogo pozitivne, uz $\lambda \geq 0$. Ovi uslovi čuvaju monotono sabiranje sadržaja lista i jednoznačno poređenje kandidata. Sve vrednosti λ koriste isti kalibracioni vektor iz jednog GPU okruženja i menjaju samo jačinu njegovog uticaja.

Za roditeljski skup S_P i kandidatske skupove S_L i S_R , cena podele glasi

$$\tilde{C}_{ponderisano} = A(P)C_T + A(L)\sum_{i \in S_L} q_i(\lambda) + A(R)\sum_{i \in S_R} q_i(\lambda). \quad (5.3)$$

Cena zadržavanja roditelja u jednom listu je

$$\tilde{C}_{list} = A(P)\sum_{i \in S_P} q_i(\lambda). \quad (5.4)$$

Odnos površina deteta i roditelja aproksimira verovatnoću ulaska, a suma težina rad koji se očekuje posle tog ulaska. Dva deteta sa jednakim brojem referenci zato mogu da dobiju različitu cenu kada sadrže različite vrste primitiva. Član C_T ostaje isti za svaku vrednost λ .

Veza sa standardnom varijantom vidi se pri $\lambda=0$:

$$\sum_{i \in S_L} q_i(0) = N_L, \sum_{i \in S_R} q_i(0) = N_R. \quad (5.5)$$

Ponderisani izraz tada postaje standardni izraz objektnog SAH modela uveden u odeljku 2.4.

U homogenom čvoru ista težina množi sadržaj svakog kandidata, pa se njihov međusobni poredak ne menja. Težina ipak može da promeni odluku između najbolje podele i zadržavanja lista, jer unutrašnji član $A^{(P)}C_T$ ostaje fiksni. Kod sfera je težina uvek jedan, dok rast težine afinih kutija može da proizvede dublje stablo.

Uz iste prefiksne i sufiksne AABB unije vode se sume q_i , pa se cena svakog kandidata i dalje dobija u konstantnom vremenu posle sortiranja. Skup osa i položaja podele jednak je standardnoj varijanti.

Izbor podela je diskretan i rekurzivan. Mala promena λ ponekad ne menja stablo, a ponekad zameni jedan kandidat i time promeni skupove primitiva u svim njegovim potomcima. Rezultati zato ne moraju da se menjaju ravnomerno sa λ .

5.4 Uticaj na topologiju

Ponderisani cilj može da nastavi podelu čvora u kome bi jedinični model izabrao list ili da izabere novu osu ili položaj podele. Tako nastalo stablo može imati više čvorova i AABB testova, ali manje skupih preseka. U heterogenoj geometriji nova topologija može i prostorno da razdvoji skupe i jeftine testove.

5.5 Ponderisane varijante srodnih metoda

Cena vrste primitive može da se kombinuje sa drugim ciljevima iz drugog poglavlja. Kod prostorne podele inspirisane metodom SBVH svaka duplirana referenca ulazi u odgovarajuću sumu q_i . Dupliranje skupe primitive zato nosi veći trošak od dupliranja sfere, dok geometrijski dobitak manjih dečjih granica ostaje u površinskom članu.

U odvojenom stablu po tipu svi članovi jednog čvora imaju istu težinu. Ona ne menja poredak prostornih kandidata sa istim C_T , ali može da promeni odluku između lista i daljeg grananja. Više korena i dalje uvodi dodatni strukturni trošak opisan u drugom poglavlju. Ponderisanje sadržaja lista ne uklanja početne AABB testove niti nezavisne putanje kroz više stabala.

Lokalne rotacije mogu da ocenjuju novu topologiju jediničnim ili ponderisanim SAH ciljem. U drugom slučaju rotacija se prihvata kada smanji očekivani rad definisan površinama i sumama q_i .

Kombinacije nisu aditivne u smislu da se dobici dve pojedinačne metode mogu sabrati. SBVH menja broj referenci i granice koje ulaze u cilj. Stablo po tipu menja broj korena i uklanjanje mešovite listove. Rotacije polaze od već izgrađene topologije. Ponderisana varijanta zato mora neposredno da se izgradi i izmeri unutar svake porodice, pa završno poređenje obuhvata četiri para standardnih i ponderisanih metoda.

5.6 Modelovani geometrijski rad

SAH procenjuje očekivanje iz granica čvora. Poseban prolaz sa brojačima daje ostvareni broj operacija za konkretan skup zraka. Iz tih brojača računa se

$$C_{work} = c_{AABB} N_{AABB} + c_s N_s + c_b N_b + c_{as} N_{as} + c_{ab} N_{ab} + c_p N_p. \quad (5.6)$$

Oznake redom predstavljaju AABB testove, sfere, kutije, affine sfere, affine kutije i ravni. Kada scena sadrži ravni, njihovi testovi pripadaju geometrijskom upitu i ostaju u zbiru, iako ih dve topologije obično izvrše podjednak broj puta.

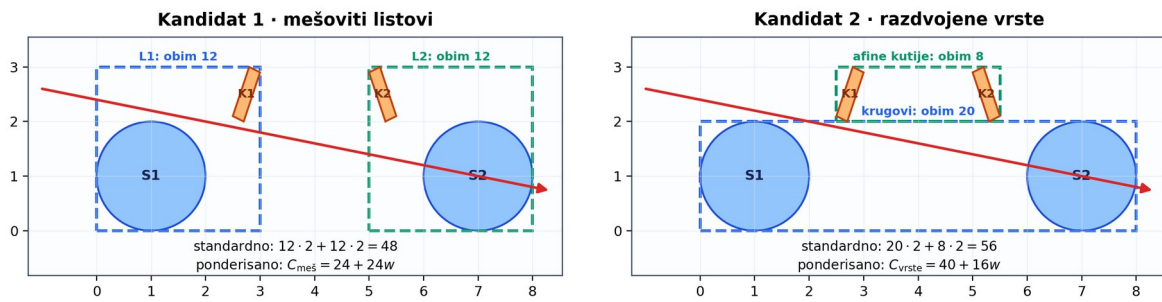
Modelovani rad pokazuje da li je smanjenje skupih testova nadoknadilo promenu AABB rada. Neposredno vreme GPU kernela dodatno obuhvata pristup memoriji, raspodelu posla po *warp* grupama i preklapanje instrukcija, koje ova suma izostavlja [10].

5.7 Geometrijski primer promene podele

Primer je dvodimenzionalni analog SAH odluke. Sadrži dva kruga koji predstavljaju sfere, poluprečnika jedan i sa centrima u $(1;1)$ i $(7;1)$, i dva tanka, zakošena paralelograma koji predstavljaju affine kutije. Njihove osno poravnate granice su $[2,5;3] \times [2;3]$ i $[5;5,5] \times [2;3]$. U ovoj ravanskoj ilustraciji obim ograničavajućeg pravougaonika ima ulogu koju površina AABB granice ima u stvarnoj trodimenzionalnoj konstrukciji.

Dvodimenzionalni analog dve SAH odluke nad istim objektima

Crvena strelica u oba panela: $r(t) = (-1; 2,6) + t(1; -0,2)$



Razdvajanje vrsta postaje povoljnije kada je $C_{vrste} < C_{meš}$, odnosno kada je $w > 2$.

Slika 5.1. Dve legalne podele istog dvodimenzionalnog skupa krugova i afinih kutija. Isprekidani osno poravnati pravougaonici predstavljaju tačne unije AABB granica objekata koje obuhvataju. Crvena strelica označava zrak koji promašuje prvi krug i obe affine kutije, a pogađa drugi krug.

Posmatrani zrak je $r(t) = (-1; 2,6) + t(1; -0,2)$. U prvom kandidatu zrak seče obe dečje granice. U bližem mešovitom listu promašuje S1 i K1, a u udaljenom promašuje K2 i pogađa S2. U drugom kandidatu zrak promašuje celu granicu lista afinih kutija i ulazi samo u list krugova, gde se ispituju preseki sa S1 i S2. Najbliži pogodak je u oba slučaja S2. Razdvajanje vrsta zato izbegava dva testa preseka sa afnim kutijama bez promene rezultata preseka.

Zajednički unutrašnji član jednak je za oba kandidata. Svako mešovito dete ima dimenzije 3×3 , obim 12 i sadrži jedan krug i jednu afnu kutiju. Promenljivi deo standardnog cilja za prvi kandidat je

$$12 \cdot 2 + 12 \cdot 2 = 48. \quad (5.7)$$

U drugom kandidatu dete krugova ima dimenzije 8×2 i obim 20. Dete afinih kutija ima dimenzije 3×1 i obim 8, pa je vrednost

$$20 \cdot 2 + 8 \cdot 2 = 56. \quad (5.8)$$

Standardni model zato bira mešoviti raspored. Ako relativnu težinu affine kutije označimo sa w , ponderisani delovi su

$$\begin{aligned} C_{meš} &= 12(1+w) + 12(1+w) = 24 + 24w, \\ C_{vrste} &= 20 \cdot 2 + 8 \cdot 2w = 40 + 16w. \end{aligned} \quad (5.9)$$

Pri $w=1$ važi $48 < 56$, pa standardni cilj bira mešovite listove. Dve vrednosti izjednačavaju se pri $w=2$, a razdvajanje vrsta postaje povoljnije za svako $w > 2$. Ista računica se u svakom čvoru koristi za poređenje kandidata međusobno i sa cenom lista.

Stvaran ishod zavisi od raspodele zraka kroz izmenjeni deo stabla. Ponderisanje je najkorisnije kada se cene funkcija jasno razlikuju i nekoliko prostornih kandidata ima sličnu vrednost standardnog cilja. Dodatna dubina može biti nepovoljna ako uvodi mnogo AABB testova, a retko izbegava skup rad u listu.

6 Eksperimentalna postavka

Četvrto i peto poglavlje opisuju standardni i ponderisani objektni SAH. Njihove razlike mogu se pripisati ciljnoj funkciji samo ako ostatak renderera ostane isti. Sve poređene konstrukcije zato koriste isti zapis scene, iste funkcije preseka, format čvorova, kamere, pseudoslučajne ulaze i GPU obilazak. Ovo poglavlje definiše kalibraciju, postavke metoda, upareni raspored merenja i način sažimanja rezultata.

6.1 Eksperimentalne celine

Evaluacija počinje odvojenim uvodnim poređenjem linearnog CPU renderera, linearnog GPU renderera i GPU renderera sa običnim objektnim BVH stablom. Njegova svrha je da razdvoji ubrzanje paralelnog izvršavanja od ubrzanja koje donosi akcelerationa struktura. Glavna evaluacija zatim ima tri povezane celine. Layered Hero ispituje osetljivost ponderisanog objektnog SAH kroz pet veličina scene, sedam veličina lista i jedanaest vrednosti λ , a zatim na istih pet scena poredi svih osam metoda. Overlap je scena sa 10.000 objekata namenski konstruisana za proveru prostornih podela pri velikom preklapanju granica. Familije F1 - F4 proveravaju svih osam metoda kroz tri veličine scene i po 100 nasumičnih prostornih rasporeda.

Uvodno poređenje posmatra ukupno vreme poziva punog renderera, jer CPU i GPU nemaju isti izvršni put. Glavna evaluacija meri vreme CUDA kernela. Za nju se odnos kandidata i reference računa dok se obe metode izvršavaju u istom CUDA kontekstu. Apsolutna vremena dobijena u različitim kontekstima ne objedinjuju se, pošto se njihove početne temperature, taktovi i troškovi inicijalizacije mogu razlikovati.

6.2 Uvodno poređenje izvršnih puteva

Uvodno poređenje koristi scene GI Test, City of Night i Signal Chamber pri 1280×720, sa jednim uzorkom po pikselu i najvećom dubinom putanje 12. Porede se tri izvršna puta: linearni CPU, linearni GPU i GPU sa običnim objektnim BVH stablom. Svaki put se pokreće u šest novih procesa. Prvi kadar u procesu služi za zagrevanje, a medijana vremena naredna tri kadra daje jedno opažanje. Konačno vreme je medijana šest procesnih opažanja.

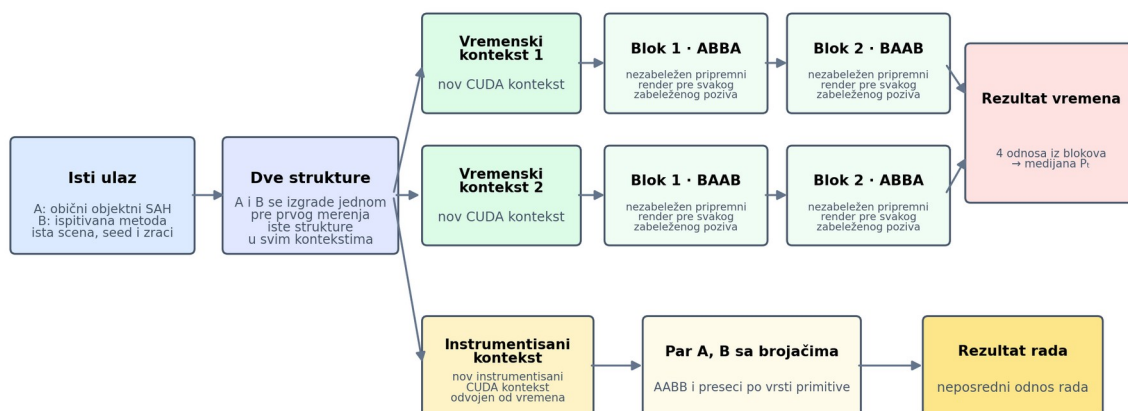
Rezultati se prikazuju u dva odvojena poređenja. Prvo poredi linearni CPU sa linearnim GPU rendererom i izdvaja uticaj paralelnog izvršavanja. Drugo zadržava GPU i poredi linearnu pretragu sa običnim objektnim BVH stablom, čime izdvaja uticaj akceleratione strukture.

6.3 Upareni redosled i provere ispravnosti

Na sledećoj slici prikazano je jedno poređenje glavne evaluacije. Oznake A i B predstavljaju referencu i kandidata, a ABBA i BAAB su dva redosleda od po četiri

zabeležena izvršavanja. Pripremni render ispred svakog slova nije uključen u vreme. Procentualni odnos računa se zasebno u svakom bloku, a zatim se uzima medijana četiri vrednosti dobijene iz blokova.

Jedno neposredno merenje kandidata B prema referenci A



Slika 6.1. Upareni postupak merenja jednog poređenja. Dva vremenska CUDA konteksta izvršavaju po jedan blok ABBA i BAAB, a treći kontekst prikuplja brojače. Svakom zabeleženom pozivu prethodi nezabeležen pripremni render iste metode.

Pre merenja proveravaju se sastav scene, binarna struktura, opsezi referenci, dozvoljena dubina steka i odsustvo prekoračenja. Referenca i kandidat unutar svakog susednog para A/B koriste isti merni seed, pa dobijaju iste piksele i iste segmente svetlosnih putanja. Instrumentisani prolazi svih metoda nad istom geometrijom takođe koriste isti seed, kako bi brojači poticali od istih svetlosnih putanja.

6.4 Glavno upareno merenje

Layered Hero, Overlap i familije F1 - F4 renderuju se pri 1920×1080, sa jednim uzorkom po pikselu i najvećom dubinom putanje 12. Primarna merena veličina je vreme CUDA kernela za ceo render. Ono ne obuhvata konstrukciju stabla, pakovanje scene, prenos na uređaj ni kopiranje završne slike.

U jednom neposrednom poređenju *A* označava obični objektni SAH, a *B* ispitivanu konstrukciju na istoj sceni. Oba se stabla grade jednom pre prvog merenja, a iste nepromenjene strukture predaju se svakom CUDA kontekstu koji ih koristi. Svaka grupa merenja izvršava se u zasebnom procesu. Taj proces jednom otvara dva neinstrumentisana vremenska konteksta i jedan instrumentisani kontekst, pa ih koristi za sva poređenja i prostorne rasporede dodeljene toj grupi.

U svakom vremenskom kontekstu izvršavaju se dva bloka, jedan ABBA i jedan BAAB. Početni redosled određuje paritet ključa merene scene ili rasporeda, rednog broja poređenja i rednog broja konteksta. Ključ razlikuje svaku Hero skalu i scenu Overlap, a u familijama F1 - F4 i konkretan prostorni raspored unutar kombinacije familije i veličine scene. Promena pariteta bilo kog od ta tri podatka zamenjuje ABBA sa BAAB. Drugi blok koristi suprotan redosled, a drugi kontekst počinje redosledom suprotnim od prvog. Menja se samo položaj reference i kandidata u bloku. Izgrađene strukture, scena, kamera i postavke renderera ostaju iste. Obe metode zato jednako često zauzimaju rane i kasne položaje.

Svakom zabeleženom izvršavanju neposredno prethodi nezabeleženi pripremni render iste metode, scene, kamere i postavki renderera. Pripremni render koristi poseban

deterministički seed. Naredni zabeleženi poziv koristi drugačiji merni seed. Pripremni render zagreva neposrednu izvršnu putanju, a njegovo vreme ne ulazi u rezultat. Isti merni seed dele samo susedni A/B pozivi unutar jednog takvog para.

Jedan blok sadrži po dva zabeležena vremena obe metode. Njegova procentualna promena računa se neposredno iz četiri sirova vremena:

$$P_T = 100 \left(1 - \sqrt{\frac{T_{B,1} T_{B,2}}{T_{A,1} T_{A,2}}} \right). \quad (6.1)$$

Svako poređenje tako daje tačno četiri procene P_T , dve iz svakog konteksta. Vrednost poređenja je njihova medijana. Odnos kandidata i reference formira se pre agregacije, čime svaki CUDA kontekst dobija jednaku težinu bez obzira na svoja apsolutna vremena.

Brojači se prikupljaju u trećem, odvojenom instrumentisanom kontekstu. Za svaku metodu izvršava se zaseban deterministički kadar punog renderera sa istom geometrijom, kamerom, rezolucijom, uzorkovanjem i najvećom dubinom kao u vremenskom merenju. Sve metode istog prostornog rasporeda dele isti seed za brojače. Taj seed koristi se samo u instrumentisanom kadru, izvan vremenskih blokova. Beleže se AABB testovi, preseki po vrsti primitive, segmenti svetlosne putanje i najveća zauzetost steka. Ako W_A i W_B označavaju modelovani rad reference i kandidata, promena rada iznosi

$$P_W = 100 \left(1 - \frac{W_B}{W_A} \right). \quad (6.2)$$

Pozitivna vrednost P_T znači kraće vreme kandidata, a pozitivna vrednost P_W manje modelovanog rada. Negativan znak označava pogoršanje u odnosu na obični objektni SAH. Vreme neinstrumentisanog kernela ostaje glavni rezultat, dok modelovani rad služi za objašnjenje promene.

Svaka Hero skala i scena Overlap koriste po jednu fiksnu geometriju za sva svoja poređenja. Na sceni Overlap svih sedam kandidata koristi isti postupak i istu referencu, obični objektni SAH.

U familijama F1 - F4 jedan prostorni raspored predstavlja jednu jedinicu opažanja. Svako od njegovih sedam poređenja prolazi kroz ista dva konteksta i četiri vremenska bloka. Rezultat svake kombinacije familije i veličine scene je medijana 100 već izračunatih procenata po rasporedu. Odnos modelovanog rada takođe se prvo računa unutar rasporeda, a zatim sažima medijanom 100 odnosa. Kombinacije familije i veličine scene ostaju odvojene.

Postupak je jednak u sve tri celine glavne evaluacije. Razlikuju se samo scena i način završne agregacije preko više nasumičnih rasporeda.

6.5 Kalibracija cena operacija

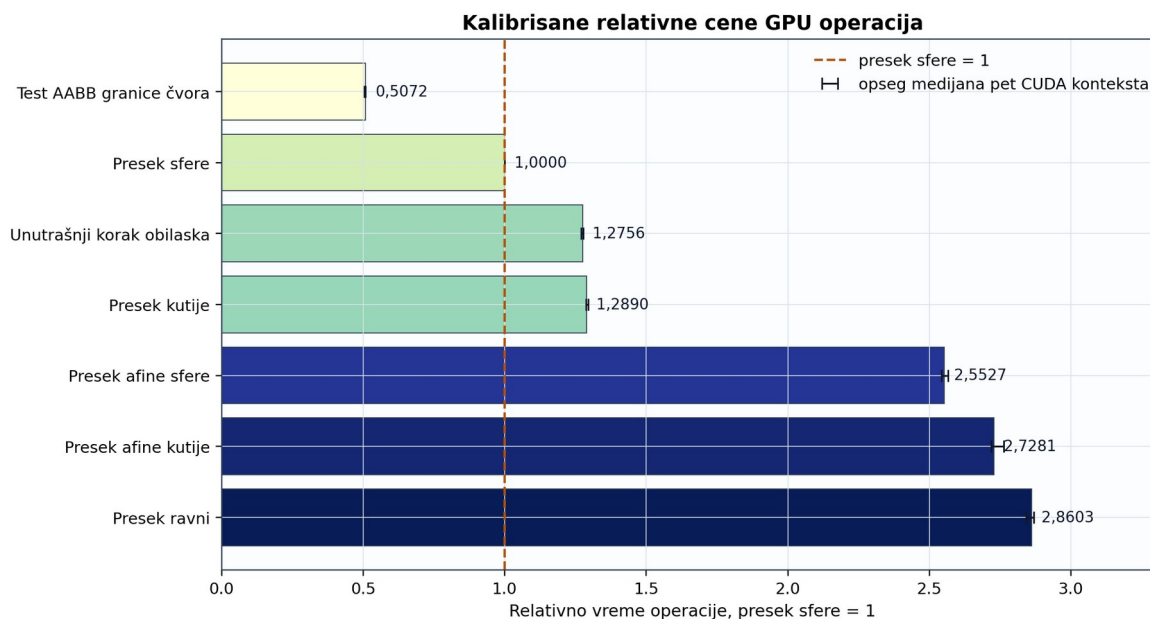
Kalibracija meri GPU funkcije renderera nad heterogenim zapisima u globalnoj memoriji. Relativne cene četiri ograničene primitive ulaze u konstrukciju, a cene AABB testa i ravni u modelovani rad. Sve su normalizovane prema preseku sa sferom.

Svaka operacija uparena je sa lokalnim merenjem sfere u pet nezavisnih CUDA konteksta. Primenjena težina je medijana pet odnosa i ista se koristi u svim završnim merenjima.

Jedan kontekst koristi 4.096 zapisa geometrije i tri deterministička profila zraka. Za svaku operaciju i profil izvršavaju se četiri para za zagrevanje i dvanaest merenih parova. Par

sadrži pripremni deo, posmatranu operaciju i lokalnu sfernu referencu u uravnoteženom redosledu.

Pripremni deo se oduzima od vremena. Odnos medijana dvanaest neto merenja operacije i uparene sfere daje procenu profila. Medijane tri profila i pet svežih konteksta daju primenjenu težinu.



Slika 6.2. Relativna vremena GPU operacija u korišćenoj kalibraciji, poređana od najkraće do najduže i normalizovana prema preseku sa sferom. Trake prikazuju primenjene medijane, a oznake raspon medijana pet CUDA konteksta.

6.6 Platforma

Merenja su izvedena na sistemu Windows sa procesorom AMD Ryzen 5 5600, 32 GB radne memorije i grafičkim procesorom NVIDIA GeForce RTX 3080. Korišćen je NVIDIA drajver verzije 610.47. Oznaka 8.6 označava računsku sposobnost GPU uređaja u CUDA modelu, odnosno nivo hardverskih mogućnosti za koji se kod izvršava.

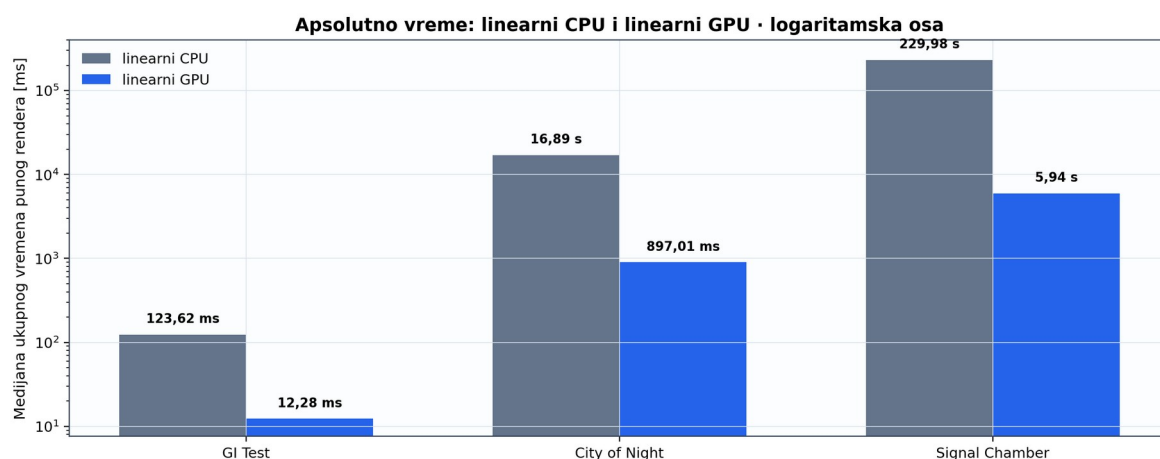
Poglavlje 7

7 Rezultati

Rezultati najpre odvajaju ubrzanje paralelnog izvršavanja od učinka akceleratione strukture. Zatim sledi analiza osetljivosti na familiji Layered Hero, predstavljanje nasumičnih familija i scene Overlap, pa zajedničko poređenje svih osam metoda. U glavnoj studiji referenca je uvek standardni objektni SAH iste scene i istih zraka. Pozitivan procenat označava kraće vreme ili manje modelovanog rada kandidata.

7.1 Linearni CPU i linearni GPU

Prvi odnos poredi isti linearni geometrijski upit u CPU i GPU rendereru, pa pokazuje učinak prelaska na paralelno izvršavanje bez uticaja BVH topologije. Meren je ukupan poziv punog renderera na scenama GI Test, City of Night i Signal Chamber, sa primarnim i sekundarnim zracima posmatranih kadrova.

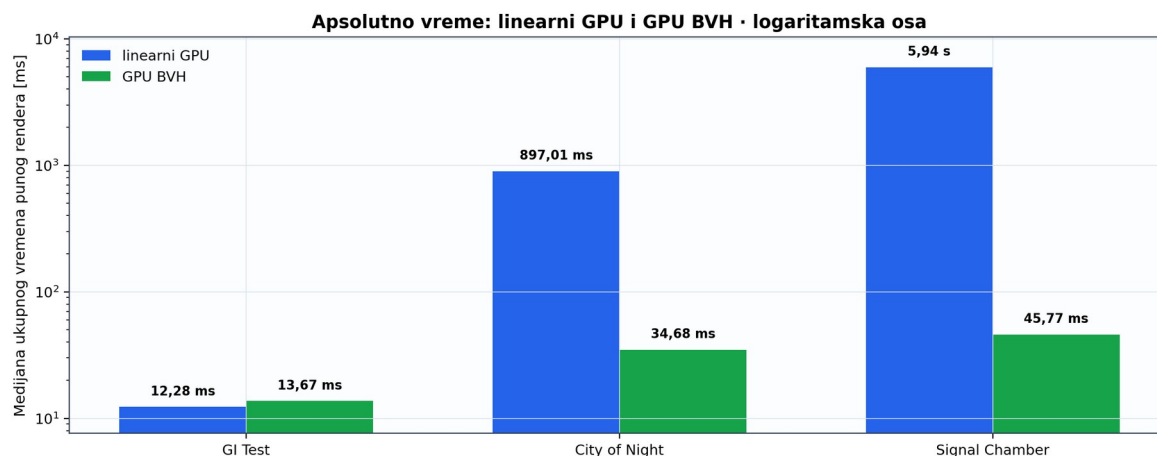


Slika 7.1. Medijane ukupnog vremena linearnog CPU renderera i linearnog GPU renderera na tri razvojne scene, prikazane na logaritamskoj osi.

Linearni GPU je brži na sve tri scene. Ubrzanje iznosi 10,06 puta na GI Test, 18,83 puta na City of Night i 38,73 puta na Signal Chamber. Razlika raste sa obimom linearne pretrage, pošto GPU istovremeno obrađuje veliki broj nezavisnih segmenata svetlosne putanje.

7.2 Linearni GPU i GPU BVH

Drugi odnos zadržava GPU izvršavanje i menja samo geometrijski upit. Linearna pretraga je referenca, a ispitivana varijanta koristi standardni objektni BVH.



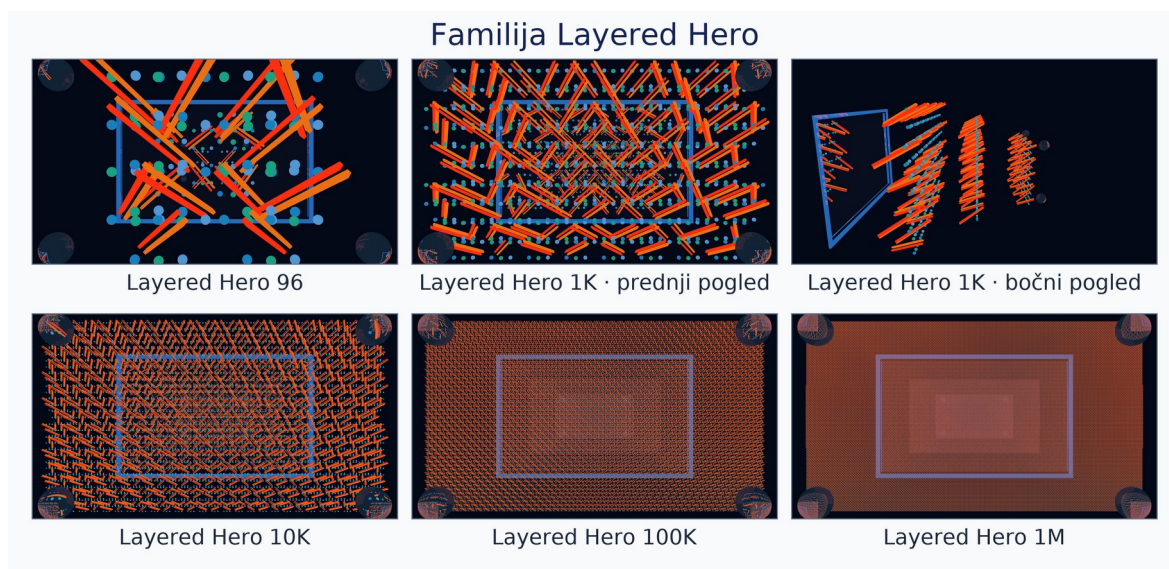
Slika 7.2. Medijane ukupnog vremena linearnog GPU renderera i GPU renderera sa standardnim objektnim BVH stablom, prikazane na logaritamskoj osi.

Na sceni GI Test odnos linearni GPU/GPU BVH iznosi 0,90, pa trošak korena, čvorova i steka blago premašuje uštedu u preseccima. Fiksni trošak BVH strukture zato nije opravdan na svakoj maloj sceni. City of Night i Signal Chamber sadrže dovoljno kandidata da BVH ukloni najveći deo linearne pretrage, uz ubrzanja od 25,87 i 129,72 puta. Signal Chamber ima približno deset puta više ograničenih primitiva od City of Night, ali vreme linearnog GPU renderera raste 6,62 puta, dok vreme renderera sa BVH strukturom raste samo 1,32 puta, što pokazuje da se renderer sa BVH strukturom mnogo bolje skalira sa povećanjem broja primitiva.

7.3 Osetljivost ponderisanog SAH na familiji Layered Hero

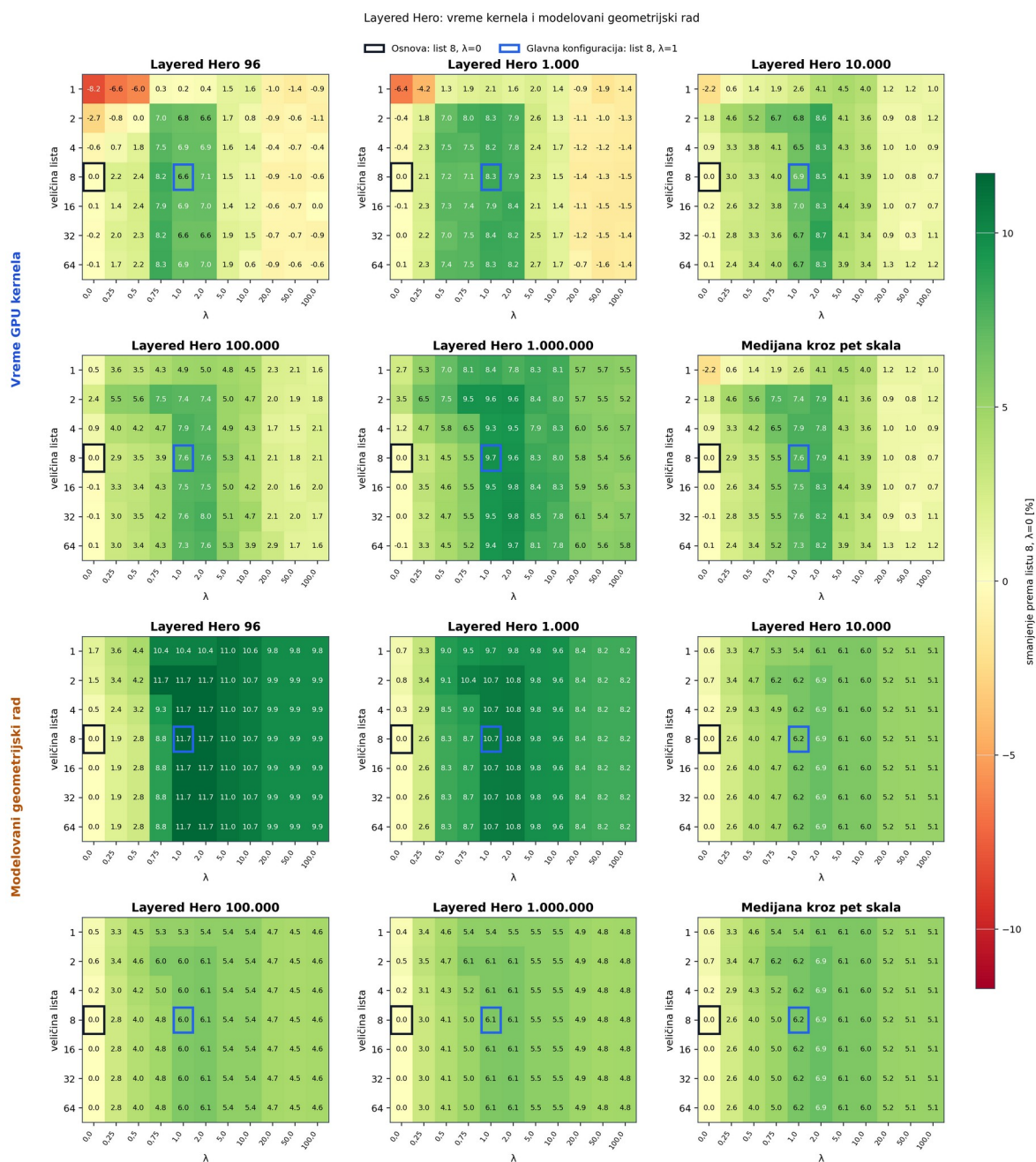
Familija Layered Hero konstruisana je tako da ponderisani cilj može da promeni izbor između više prostorno prihvatljivih podela. Ponavljani modul sadrži šest sfera i dve tanke affine kutije raspoređene u tri smaknuta dubinska sloja. Time isti mali prostorni obrazac kombinuje češće, jeftinije preseke sa sferama i ređe, skuplje preseke sa afinim kutijama. Prednji pogled i sastav modula ostaju jednaki kroz skale, dok bočni pogled pokazuje slojeve i njihovu strukturu.

Promenljivi deo sadrži 96, 1.000, 10.000, 100.000 i 1.000.000 primitiva. Stalni deo kadra čine zadnje ogledalo u okviru i četiri manje dekorativne staklene sfere. Oni se ne ubrajaju u navedene veličine. Ogledalo i dekorativne sfere prikazuju refleksije i prelamanja tokom punog praćenja putanja, a ponovljeni moduli ostaju deo scene čiji se obim menja.



Slika 7.3. Prikazi familije Layered Hero od 96 do jednog miliona skalabilnih primitiva. Prednji pogledi dokumentuju isti odnos sfera i afinih kutija kroz pet skala, a bočni pogled reprezentativne scene pokazuje tri dubinska sloja.

Za svaku skalu ispituje se sedam veličina lista i jedanaest vrednosti λ . Referentna ćelija koristi veličinu lista 8 i $\lambda=0$, a glavno ponderisano podešavanje veličinu lista 8 i $\lambda=1$. Ostale ćelije neposredno su uparene sa referencom iste skale. Gornja matrica prikazuje procentualnu promenu vremena izvedenu iz uparenih merenja, a donja modelovani geometrijski rad. Razlika između matrica izdvaja slučajeve u kojima promena kalibrisanih geometrijskih operacija ne objašnjava potpuno vreme GPU kernela.



Slika 7.4. Osetljivost familije Layered Hero na veličinu lista i parametar ponderisanja. Gornji deo prikazuje neposredno izmerenu procentualnu promenu vremena, a donji promenu modelovanog geometrijskog rada prema standardnom konfiguraciji sa veličinom lista 8 iste skale. Crni okvir označava veličinu lista 8, $\lambda = 0$, a plavi veličinu lista 8, $\lambda = 1$.

Iz rezultata se vidi da već blago ponderisanje sa $\lambda = 0,25$ ili $\lambda = 0,5$ skraćuje vreme na svih pet skala. Medijansko smanjenje vremena raste do $\lambda = 2$. Modelovani rad je manji na svih pet skala za svako $\lambda > 0$, a najveća medijana smanjenja postiže se pri $\lambda = 2$. Pri veličini lista 8 najbolju medijanu kroz celu familiju daje $\lambda = 2$, sa skraćanjem vremena od 7,91%, što ukazuje da ovom pravilnom rasporedu pogoduje jače razlikovanje skupih afinih kutija i jeftinih sfera. Povećanje parametra na $\lambda = 50$ ili $\lambda = 100$ produžava vreme na dve najmanje skale iako modelovani rad ostaje manji, što pokazuje da previše agresivno ponderisanje može da bude štetno. Veličina lista 1 daje najslabiji rezultat na svih pet skala, dok su medijane ostalih veličina bliske.

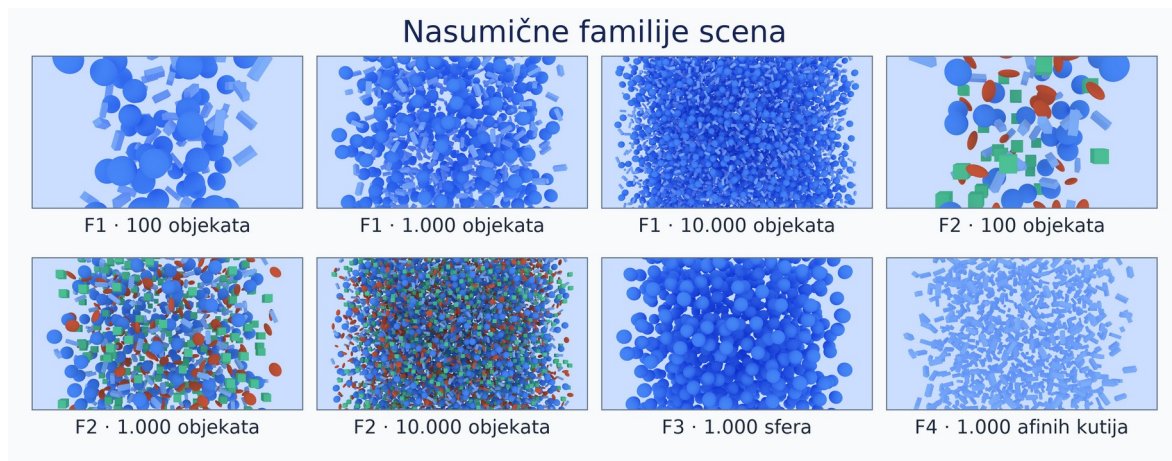
Za zajedničko poređenje metoda koristi se $\lambda=1$, jer neposredno primenjuje kalibrisane relativne cene. Veće vrednosti ostaju deo analize osetljivosti i ne menjaju glavno podešavanje ostalih metoda.

7.4 Scene završnog poređenja

7.4.1 Nasumične familije F1 - F4

Familije F1 - F4 proveravaju ponašanje metoda izvan pravilnih Hero modula. Za svaku kombinaciju familije i veličine scene od 100, 1.000 ili 10.000 objekata koristi se 100 rasporeda. Odgovarajući rasporedi dele centre objekata između različitih familija, čime se sastav primitiva menja uz očuvanu prostornu organizaciju.

F1 meša sfere i affine kutije, a F2 sva četiri ograničena tipa. Ove dve familije proveravaju heterogene listove i trošak više stabala po tipu. F3 sadrži sfere jedinične kalibrisane cene i služi kao nulta kontrola. Ponderisani i standardni objektni SAH tada imaju isti cilj. F4 sadrži samo affine kutije. Njihova nejedinična cena proverava da li ponderisanje menja odluku između lista i dalje podele i u homogenoj sceni.

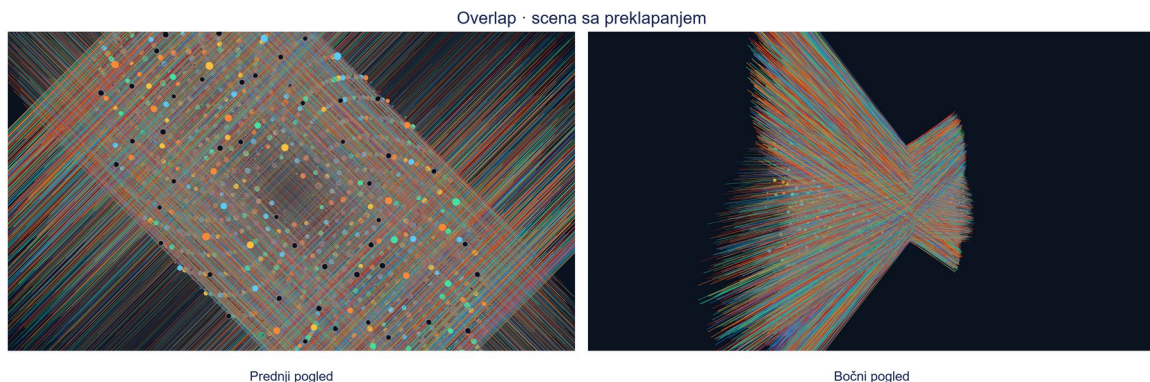


Slika 7.5. Reprezentativni rasporedi familija F1 - F4. F1 i F2 sadrže heterogene populacije, F3 sfere, a F4 affine kutije.

7.4.2 Scena Overlap

Overlap sadrži 3.000 sfera i 7.000 dugih, veoma tankih afinih kutija u ukrštenom rasporedu. Jedan kraj duge ose svake kutije nalazi se bliže kameri, a drugi dublje u sceni. Prednji i bočni pogled zajedno pokazuju da se AABB granice tih kutija snažno preklapaju duž sve tri prostorne ose.

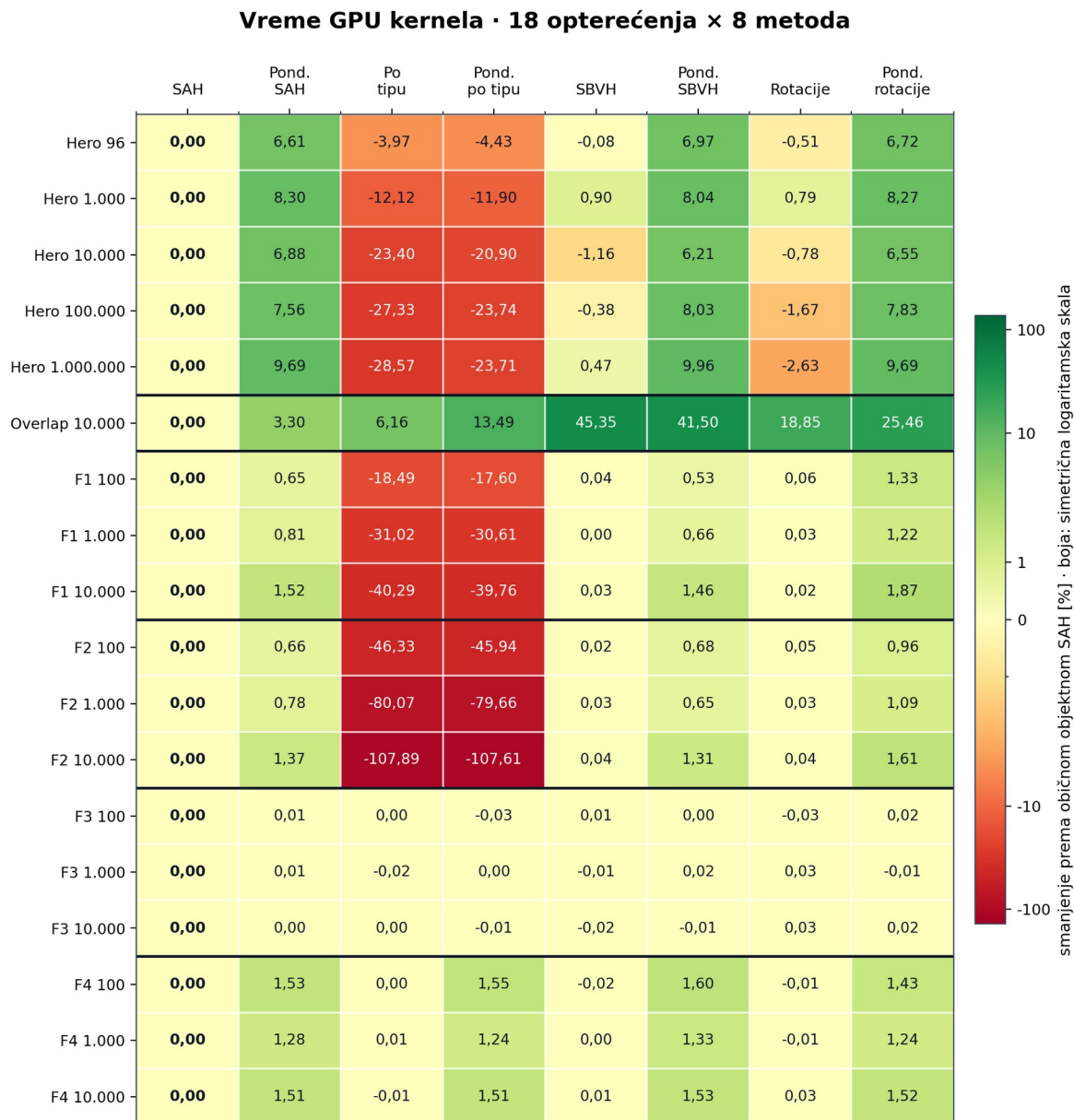
Ova scena namenski proverava prostorne podele. Objektni SAH mora celu granicu primitive da dodeli jednom detetu, dok SBVH može da duplira referencu i odseče njenu granicu na ravni podele. Prednji pogled koristi se u merenju, a bočni prikazuje promenu dubine duž njihovih dugih osa.



Slika 7.6. Scena Overlap sa 3.000 sfera i 7.000 tankih afinih kutija.

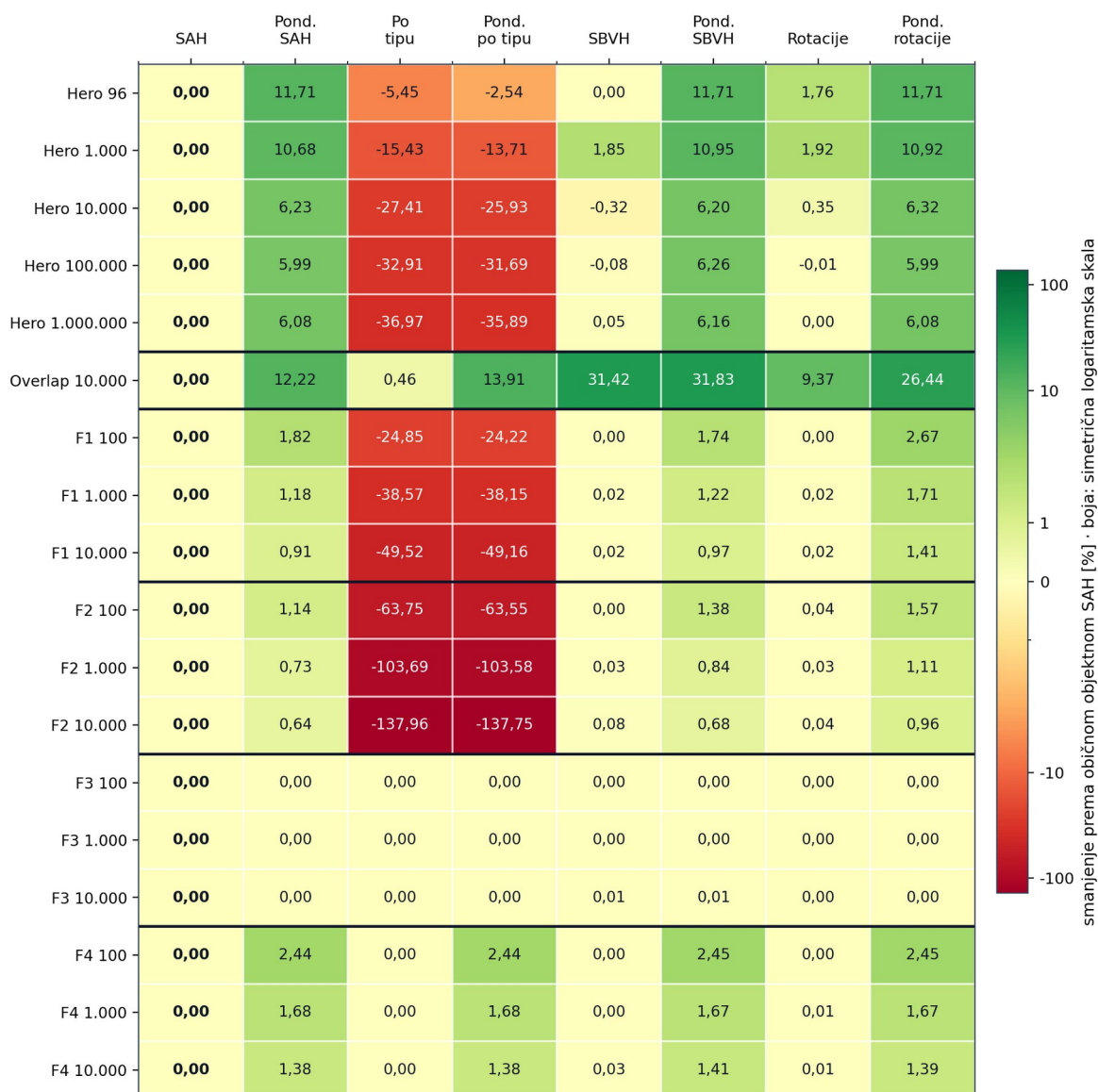
7.5 Rezultati svih osam metoda

Zajednički skup obuhvata standardne i ponderisane varijante objektnog SAH, stabala po tipu, SBVH i lokalnih rotacija. Standardni objektni SAH ima vrednost nula u svakom redu. Svaki od preostalih sedam rezultata dobijen je neposrednim poređenjem sa tom referencom na istoj geometriji i istim zracima. Međusobni odnosi dva kandidata izvode se preko njihove zajedničke reference. Pet Hero skala, scena Overlap i dvanaest F kombinacija analiziraju se odvojeno. Detaljni opisi četiri porodice metoda dati su u odeljku 2.5.



Slika 7.7. Procentualna promena vremena GPU kernela svih osam metoda prema standardnom objektnom SAH iste scene. Matrica obuhvata pet Hero skala, scenu Overlap i svih dvanaest F kombinacija.

Modelovani geometrijski rad · 18 opterećenja × 8 metoda



Slika 7.8. Procentualna promena modelovanog geometrijskog rada svih osam metoda prema standardnom objektom SAH iste scene, u istom rasporedu redova i kolona kao matrica vremena.

7.5.1 Layered Hero

Rezultati u pet Hero redova prikazuju ishod svih osam metoda pri izabranoj veličini lista 8. Za četiri ponderisane varijante postavljeno je $\lambda=1$. Standardni i ponderisani par svake porodice koristi isti format čvora i obilazak, pa njihova razlika izdvaja uticaj kalibrisane cene sadržaja lista unutar te porodice.

Analiza osetljivosti već je pokazala da ponderisani objektni SAH skraćuje vreme na svih pet Hero skala, uz medijanu od 7,56%.

Kod metode koja gradi zasebno BVH stablo za svaki tip, sfere i affine kutije pripadaju različitim korenima. Odnos ove metode prema standardnoj konstrukciji pogoršava se sa rastom scene. Ponderisanje popravljaja njeno vreme na četiri od pet skala, uz medijanski dobitak od 2,03% prema standardnoj varijanti, ali trošak dva odvojena obilaska i dalje nadmašuje korist homogenih listova.

Najveću medijanu skraćenja vremena, 8,03%, ostvaruje ponderisani SBVH. Obični SBVH nema pozitivan medijanski rezultat na ovoj familiji, dok ponderisana varijanta menja ishod.

Medijanski rezultat običnih rotacija produžava vreme za 0,78% i smanjuje modelovani rad za 0,35%. Ponderisane rotacije dostižu medijane od 7,83% za vreme i 6,32% za modelovani rad, malo iznad ponderisanog objektnog SAH po obe mere. Rezultat pokazuje da lokalno poboljšanje standardnog cilja ne mora da smanji vreme, dok rotacije nad ponderisanom topologijom po medijani daju mali dodatni dobitak.

7.5.2 Scena Overlap

Scena Overlap napravljena je da istakne korist prostornih podela, a SBVH na njoj daje najveće skraćanje vremena, 45,35%. Ponderisani SBVH smanjuje modelovani rad za dodatnih 0,61% prema standardnom SBVH, ali vreme mu je 7,05% duže.

Razlika vremena i modelovanog rada vidi se i kod stabala po tipu. Obična stabla po tipu brža su od ponderisanog objektnog SAH, a ponderisana stabla po tipu dodatno poboljšavaju rezultat i imaju najbolji ishod među te tri metode. Modelovani rad takođe izdvaja metodu ponderisanih stabala po tipu kao najbolju od te tri metode, dok obična stabla po tipu prema toj meri imaju znatno slabiji rezultat nego prema vremenu. Moguće objašnjenje za dobar rezultat stabala po tipu na ovoj sceni jeste to što se male sfere efikasnije organizuju u zasebnom stablu kada duge kutije ne proširuju njihove zajedničke granice.

Lokalne rotacije takođe daju veliki dobitak na ovoj sceni. Njihov rezultat pokazuje da početna konstrukcija zasnovana na centroidnim podelama ostavlja prostor za naknadno lokalno poboljšanje topologije. Velike AABB granice tankih, izduženih kutija pokazuju da centroid sam po sebi slabo opisuje njihov prostorni zahvat, pa rotacije mogu da poprave deo odluka zasnovanih na centroidnim podelama.

7.5.3 Familije F1 - F4

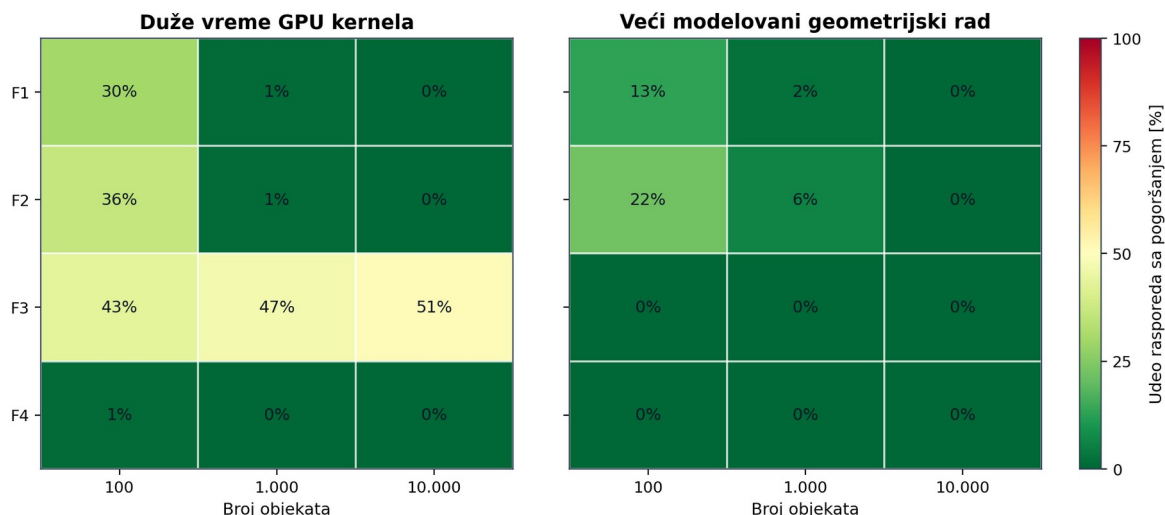
Svaki red familija F predstavlja medijanu 100 neposrednih poređenja sa standardnim objektnim SAH. F1 i F2 proveravaju da li se učinak ponderisanja održava na nepravilnim heterogenim rasporedima i kako se menja sa veličinom scene.

Stabla po tipu daju znatno duže vreme u heterogenim familijama F1 i F2, kao i na familiji Layered Hero. Trošak odvojenih korena u tim rasporedima nadmašuje korist homogenih listova.

U kontrolnoj familiji F3 standardni i ponderisani cilj daju iste topologije i isti modelovani rad. Preostale male promene vremena predstavljaju mernu varijaciju.

U homogenoj familiji F4 ponderisanje smanjuje vreme svake od četiri metode na sve tri veličine scene. Nejedinična cena afine kutije zato utiče i bez mešanja tipova, kroz odnos cene lista i nastavka podele.

Medijana ne pokazuje u koliko je pojedinačnih rasporeda rezultat imao negativan znak. Sledeća slika zato za ponderisani objektni SAH prikazuje udeo rasporeda u kojima su vreme ili modelovani rad lošiji nego kod standardnog objektnog SAH. Svaka ćelija odgovara jednoj kombinaciji familije i veličine scene i obuhvata 100 rasporeda.

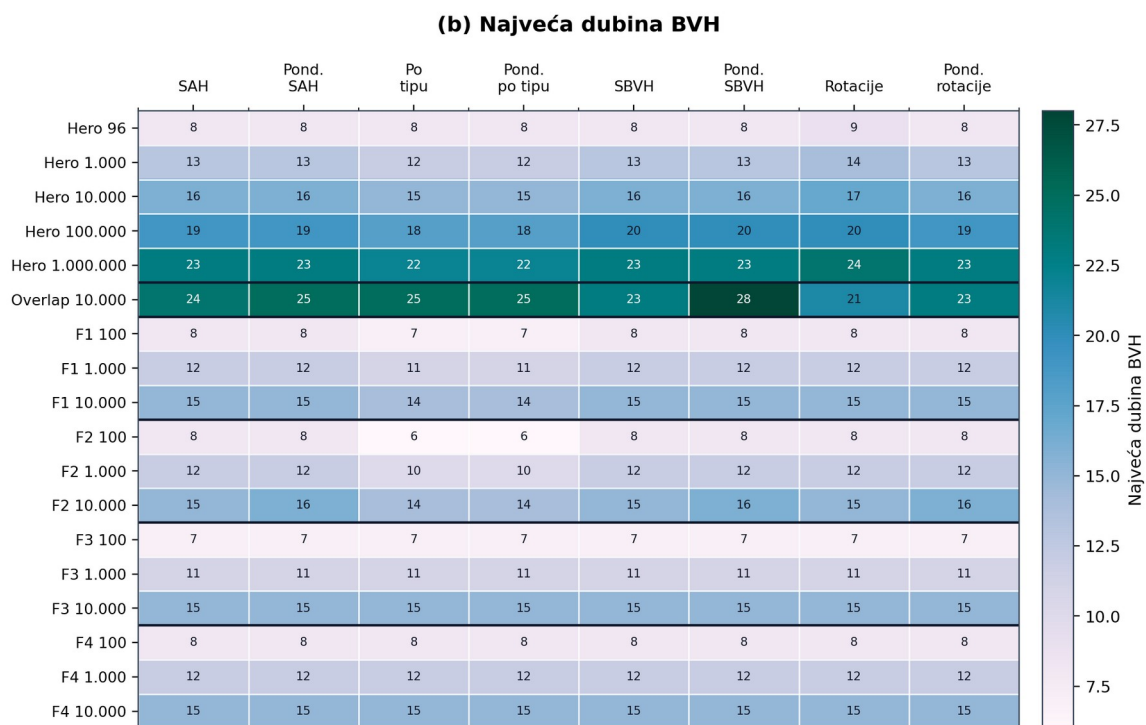
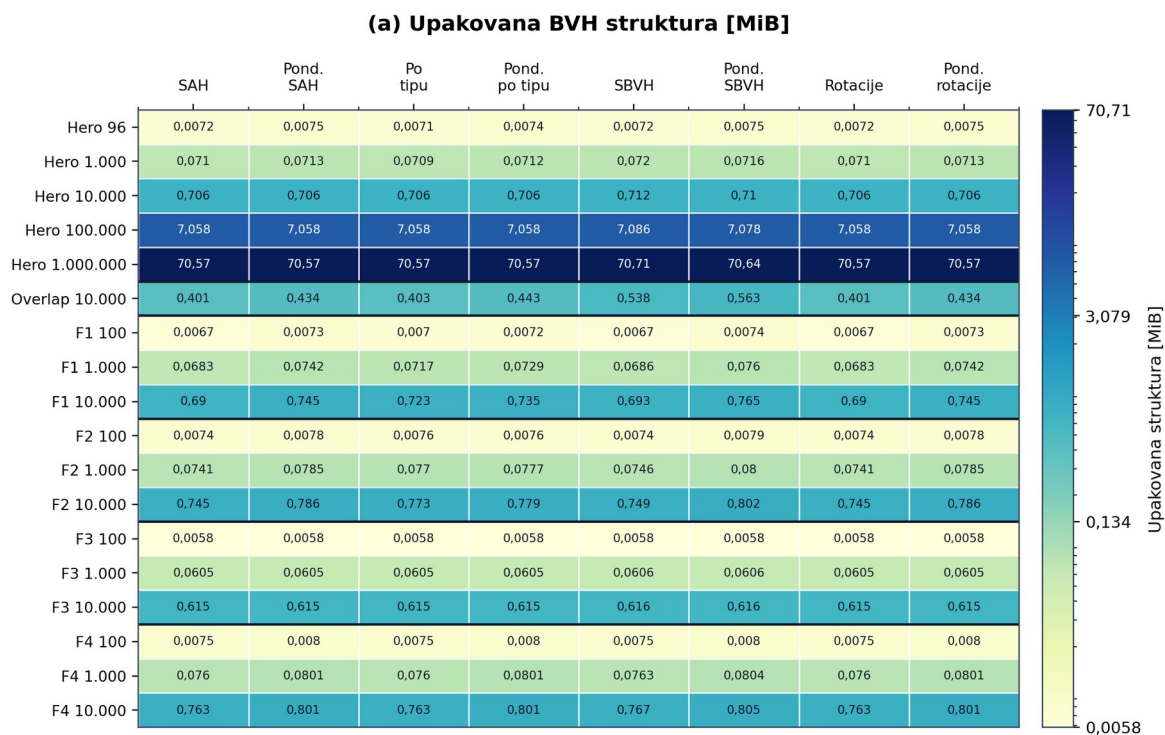


Slika 7.9. Udeo od 100 nasumičnih rasporeda u kojima ponderisani objektni SAH ima duže vreme GPU kernela ili veći modelovani geometrijski rad od standardnog objektnog SAH. Svaka ćelija predstavlja jednu kombinaciju familije i broja objekata.

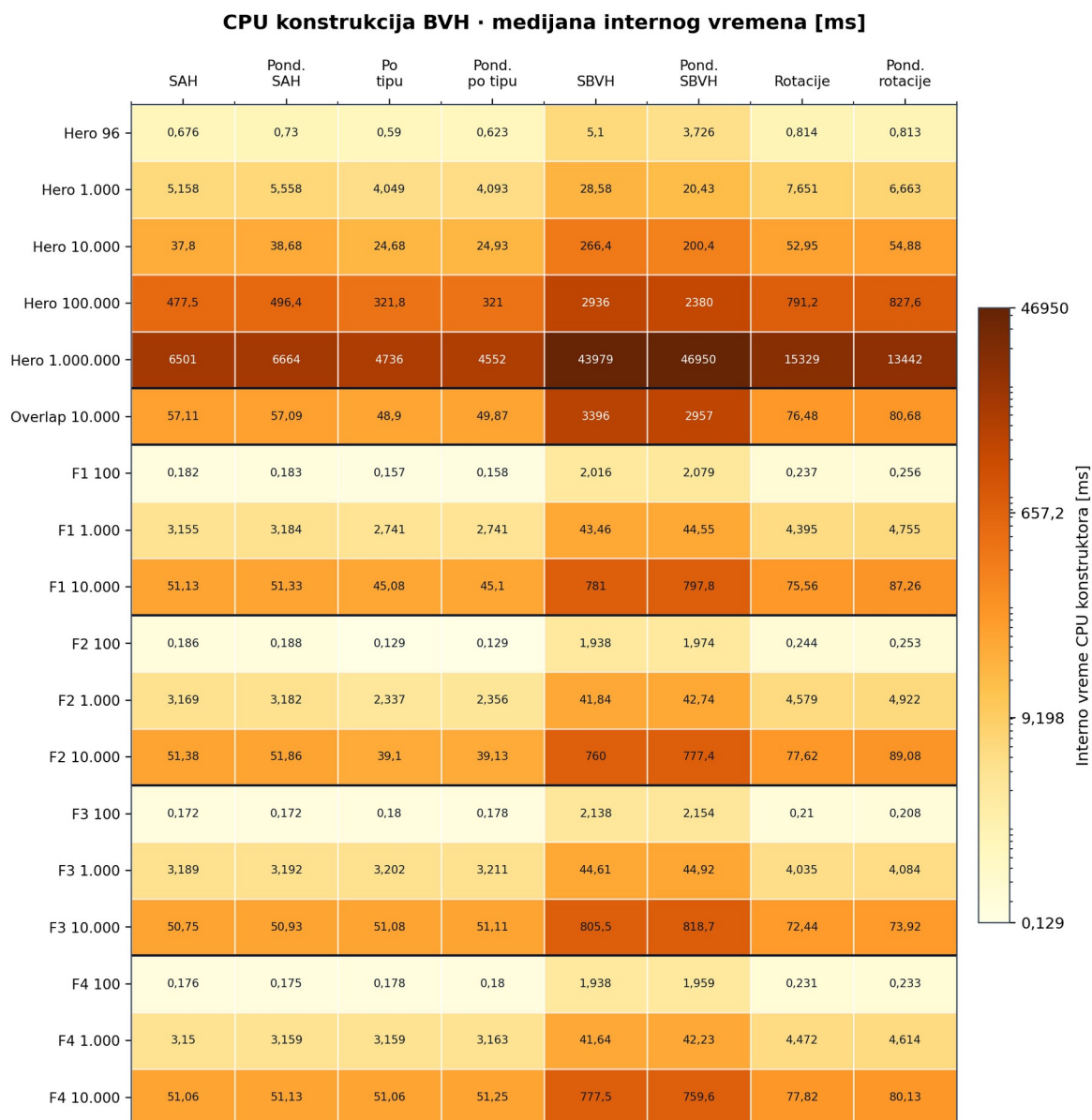
U familijama sa merljivim efektom, F1, F2 i F4, duže vreme GPU kernela javlja se u 22,33% rasporeda sa 100 objekata, u 0,67% rasporeda sa 1.000 objekata i ni u jednom rasporedu sa 10.000 objekata. Takvi slučajevi znatno su češći na scenama sa 100 objekata, gde ponderisana topologija ne daje prednost u svakom rasporedu, a sa rastom scene takvi slučajevi postaju ređi. Povećanje modelovanog rada javlja se u manjem ukupnom broju rasporeda i takođe nestaje pri 10.000 objekata. F3 ostaje odvojena nulta kontrola. Standardni i ponderisani cilj daju isto stablo, a modelovani rad je jednak u svih 100 rasporeda na svakoj skali, pa njegova promena iznosi 0%. Mala vremenska odstupanja predstavljaju mernu varijaciju.

7.6 Veličina, dubina i vreme konstrukcije

Strukturne mere odnose se na iste BVH strukture koje su korišćene u matricama vremena i modelovanog rada. Za nasumične familije svaka ćelija prikazuje medijanu kroz 100 rasporeda. Veličina obuhvata spakovane podatke potrebne GPU obilasku, a dubina je najveća strukturna dubina izgrađenog stabla ili šume.



Slika 7.10. Spakovana veličina BVH strukture u MiB i najveća strukturna dubina svih osam metoda na Hero skalama, sceni Overlap i familijama F1 - F4. Čelije nasumičnih familija prikazuju medijane kroz 100 rasporeda.



Slika 7.11. Vreme CPU konstrukcije svih osam metoda za strukture korišćene u završnom poređenju. Čelije nasumičnih familija prikazuju medijane kroz 100 rasporeda.

Vreme konstrukcije opisuje determinističku CPU implementaciju namenjenu potpunom i kontrolisanom poređenju kandidata. Brzina izgradnje nije optimizovana, pa vrednosti služe relativnom poređenju ovih implementacija. Optimizovani paralelni konstruktori zahtevaju zasebnu studiju. Konstrukcija se nalazi van vremena GPU kernela iz prethodnih matrica.

7.6.1 Layered Hero

Svih osam metoda koristi isti format čvora. Spakovana veličina zavisi od broja čvorova, korena i referenci na primitive, dok dubina zavisi od rasporeda čvorova u topologiji. Objektna metoda ne dupliraju reference, pa razlike u njihovoj veličini potiču od broja korena, listova i čvorova. SBVH može dodatno da poveća veličinu strukture dupliranjem referenci. Vreme konstrukcije obuhvata ceo postupak svake metode, uključujući naknadne prolaze rotacija i obradu prostornih kandidata kod SBVH.

Dubina svih metoda na Hero skalama ostaje jednaka ili se od standardnog objektnog SAH razlikuje za jedan nivo. U slojevitoj geometriji ove scene SBVH duplira samo mali deo referenci, pa medijansko povećanje veličine iznosi 0,41% za SBVH i 0,62% za ponderisani SBVH. Veće razlike javljaju se u konstrukciji. Stabla po tipu skraćuju njeno vreme za

27,16% do 29,99%, ponderisani objektni SAH ga povećava za 3,95%, a rotacije za 45,19% do 48,33%. Odnos vremena konstrukcije dve SBVH varijante prema vremenu standardne konstrukcije kreće se od 5,30 do 6,76. Njegova konstrukcija dodatno ocenjuje prostorne kandidate, odseca granice, upravlja duplikatima i primenjuje pohlepno uklanjanje duplikata.

7.6.2 Scena Overlap

SBVH može da duplira referencu kada prostorna podela preseče granicu primitive. Smanjenje preklapanja zato može da prati veća struktura ili dublje stablo. Matrice standardnog i ponderisanog SBVH povezuju taj strukturni trošak sa odnosom AABB testova i testova primitiva iz prethodnog odeljka. Vreme konstrukcije obuhvata razmatranje prostornih kandidata i odsecanje granica.

SBVH strukture na Overlap rastu mnogo više nego na Hero, što ukazuje na češće prostorne podele. Struktura SBVH je 34,18% veća od standardne, a ponderisana varijanta 40,48% veća i dostiže dubinu 28 prema 23 kod običnog SBVH. Ponderisane varijante objektnog SAH, SBVH i rotacija takođe su dublje od svojih standardnih parova. Odnosi vremena konstrukcije običnog i ponderisanog SBVH prema vremenu standardne konstrukcije iznose približno 59,47, odnosno 51,78. Visok trošak konstrukcije prati porodicu SBVH, čija standardna varijanta daje najbolje vreme kernela na ovoj sceni. Ukupna isplativost nije merena i zavisi i od broja rendera kroz koje se konstrukcija amortizuje.

7.6.3 Familije F1 - F4

Medijane nasumičnih rasporeda prikazuju strukturu svake veličine scene. U F1 i F2 stabla po tipu imaju dva ili četiri nezavisna korena, dok svaka od ostalih metoda gradi sopstvenu jednokorensku hijerarhiju. F3 služi kao strukturna kontrola jer standardni i ponderisani cilj donose iste odluke, a F4 proverava da li nejedinična cena primitive menja dubinu i veličinu bez mešovityh listova.

Na nasumičnim familijama dubine većine metoda ostaju jednake ili bliske standardnoj konstrukciji, dok su stabla po tipu plića u F1 i F2. Medijanska veličina ponderisanog objektnog SAH stabla veća je za 5,74%, a ponderisanog SBVH za 6,67%, u odnosu na standardni objektni SAH. Vreme konstrukcije ponderisanog objektnog SAH ostaje gotovo jednako, sa promenom od 0,37%, dok ga stabla po tipu skraćuju za 5,71% do 5,92%. Rotacije povećavaju vreme za približno 40,63% do 46,06%. Odnosi vremena konstrukcije običnog i ponderisanog SBVH prema vremenu standardne konstrukcije iznose 13,50, odnosno 13,79, pa njihova specijalizovana prostorna pretraga nosi jasan pripremni trošak i kada veličina stabla ostane bliska ostalim metodama.

Poglavlje 8

8 Zaključak

U radu je razvijena i eksperimentalno ispitana BVH konstrukcija za scene sa analitičkim primitivama čiji testovi preseka nemaju jednaku cenu na grafičkom procesoru. Ponderisani SAH cenu lista računa kao zbir kalibrisanih relativnih cena konkretnih vrsta primitiva. Tokom konstrukcije menja se procena sadržaja lista, dok format stabla i GPU obilazak ostaju nepromenjeni.

8.1 Odgovori na istraživačka pitanja

Prvo pitanje. Koliko se razlikuju cene GPU funkcija preseka za podržane vrste primitiva i kako se te razlike mogu ugraditi u cenu lista?

Mikromerenje je potvrdilo da implementirani GPU testovi preseka imaju različite relativne cene. Njihovo uvođenje u član lista omogućava konstruktoru da razlikuje podele koje standardni broj referenci ocenjuje jednako, a u obilasku ostavljaju različit geometrijski rad. Kalibracija pripada korišćenom kodu i uređaju, pa je treba ponoviti kada se promeni implementacija preseka ili GPU platforma.

Drugo pitanje. Da li ponderisani objektni SAH smanjuje vreme GPU kernela za ceo render na namenski konstruisanoj familiji Layered Hero i na nasumičnim scenama različitog sastava?

Ponderisani objektni SAH skraćuje vreme na svih pet Hero skala, uz medijanu od 7,56%. Kroz svih dvanaest F kombinacija medijana iznosi 0,80%. Svih devet kombinacija familija F1, F2 i F4 ima kraće medijansko vreme. Preostale tri kombinacije pripadaju kontrolnoj familiji F3. Standardni i ponderisani cilj u njima daju iste topologije i isti modelovani rad, pa mala vremenska odstupanja predstavljaju mernu varijaciju. Efekat u F1, F2 i F4 manji je nego na Layered Hero, a na pojedinačnim rasporedima vreme može biti duže.

Layered Hero izdvaja ponašanje ponderisanog cilja u pravilnoj heterogenoj geometriji, dok nasumični rasporedi proveravaju koliko se isti mehanizam prenosi na različite prostorne odnose i sastave primitiva. Upareno vreme ostaje merilo učinka, a modelovani rad služi za objašnjenje promene topologije.

Treće pitanje. Kako se ponderisanje ponaša u stablima po tipu, prostornim podelama i lokalnim rotacijama, naročito na sceni projektovanoj za jako preklapanje?

U većini ispitanih scena ponderisana varijanta svake od četiri porodice daje kraće vreme od odgovarajuće standardne varijante. Ponderisani SBVH ima najbolju Hero medijanu, obični SBVH pobeđuje na sceni Overlap, a ponderisane rotacije daju najbolje porodične medijane na F1 i F2. Rezultati zato ne izdvajaju jednu varijantu za sve scene.

Stabla po tipu donose homogene listove po cenu više korena i nezavisnih obilazaka. SBVH menja prostor kandidata i može da duplira reference, dok lokalne rotacije polaze od već izgrađene topologije. Zbog tih razlika učinak ponderisanja tumači se unutar svake porodice konstruktora.

8.2 Praktični značaj

Praktična vrednost ponderisanog objektnog SAH potiče iz male promene postojećeg sistema. GPU format i obilazak ostaju isti, vreme konstrukcije je gotovo jednako vremenu konstrukcije standardne varijante, a nova obaveza je kalibracija funkcija preseka za konkretni kod i uređaj. Metoda zato odgovara statičnim scenama sa heterogenim analitičkim primitivama u kojima je poželjno zadržati postojeći GPU izvršni put.

Razvijene testne scene imaju zasebne dijagnostičke uloge. Layered Hero izdvaja uticaj jačine ponderisanja i veličine lista kroz rast pravilne scene. Familije F1 - F4 proveravaju prenos rezultata na nasumične rasporede i uvode homogene kontrole, dok Overlap izdvaja geometriju na kojoj prostorne podele imaju jasnu prednost. Takva podela testova olakšava povezivanje rezultata sa osobinama scene.

Rezultati ne izdvajaju jedan konstruktor za sve rasporede. SBVH daje najveću korist na sceni sa snažnim preklapanjem, dok ponderisani objektni SAH pruža stabilan dobitak na Layered Hero uz najmanju promenu implementacije. Upareni postupak i kontrolisane familije scena mogu se ponovo koristiti za procenu novih konstruktora i drugih vrsta primitiva.

8.3 Ograničenja i dalji rad

8.3.1 Validnost merenja

Temperatura uređaja, promena takta i aktivnost operativnog sistema ostaju izvori varijacije kratkih GPU merenja. Neposredno uparivanje kandidata i reference, suprotni redosledi u odvojenim CUDA kontekstima i pripremni renderi smanjuju sistematski uticaj redosleda. Ista izgrađena struktura koristi se u vremenskim i instrumentisanim kontekstima. Ove mere ograničavaju šum, ali ga ne uklanjaju iz pojedinačnog poređenja.

Vreme GPU kernela obuhvata deo renderera koji izvršava praćenje putanja i BVH obilazak. Konstrukcija stabla, slanje scene i stabla na GPU i kopiranje završne slike nalaze se izvan te mere. Vreme CPU konstrukcije i veličina strukture u sedmom poglavlju imaju opisnu ulogu. Korišćeni konstruktori razvijeni su za kontrolisano poređenje ciljne funkcije. Memorijski otisak, brzina izgradnje, prenos i ažuriranje nisu bili zasebni ciljevi optimizacije.

Modelovani geometrijski rad obuhvata AABB testove i testove primitiva ponderisane kalibrisanim cenama. Memorijski promet, koalesiranje, zauzetost izvršnih jedinica, stanje steka i raspored instrukcija ostaju objedinjeni u neposredno izmerenom vremenu. Model služi za tumačenje mehanizma, a vreme kernela određuje učinak.

8.3.2 Opseg zaključaka

Evaluacija obuhvata namenski konstruisanu familiju Layered Hero, scenu Overlap i četiri familije nasumičnih rasporeda. Te celine imaju različite jedinice opažanja i analizirane su odvojeno. Broj rasporeda sa pogoršanjem dopunjuje medijanu nasumične familije i sprečava da se njen znak automatski pripiše svakom pojedinačnom prostornom rasporedu. Dodatne proceduralne familije i realne scene mogu da otkriju drugačije odnose geometrije, kamere i sekundarnih putanja, a postojeća nasumična studija pokazuje da pozitivan zbirni rezultat ne garantuje poboljšanje svakog rasporeda.

Kalibracija i merenja izvedeni su na RTX 3080, sa četiri vrste ograničenih analitičkih primitiva, binarnom softverskom BVH strukturom i odabranim kamerama. Trouglovi, široka stabla, hardverske jedinice za praćenje zraka, animirana geometrija i drugačije raspodele sekundarnih putanja ostaju van ispitnog opsega.

8.3.3 *Pravci daljeg rada*

Prvi korak je ponavljanje kalibracije i završnih poređenja na drugim GPU arhitekturama. Trouglovi, široka stabla i hardversko praćenje zraka proširili bi skup primitiva i načina obilaska na kojima se proverava ponderisani cilj. Hardversko profilisanje memorijskog prometa i izvršavanja grupa niti može da razdvoji slučajeve u kojima se vreme i modelovani rad ne menjaju istim smerom ili intenzitetom.

Za dinamične scene potrebno je meriti ukupnu cenu konstrukcije, prenosa, ažuriranja i obilaska kroz niz kadrova. Paralelna HLBVH konstrukcija zasnovana na radnim redovima [32] i efikasno ažuriranje BVH strukture za animirane scene [33] daju relevantne polazne tačke za takvo poređenje. Time bi se moglo odrediti kada poboljšanje obilaska nadoknađuje cenu izgradnje ili ažuriranja ponderisane strukture.

Ponderisanje po tipu može se povezati i sa ciljevima koji procenjuju raspodelu zraka ili vidljivost, kao što su RDH [15] i varijanta koju autori označavaju kao OSAH [16]. Takvo proširenje zahteva zajedničko ispitivanje cene tipa primitive i procenjene verovatnoće obilaska u punom rendereru za praćenje putanja.

Biografija

Nikola Pešić rođen je 2001. godine u Novom Sadu. Od ranog uzrasta interesovao se za matematiku i rešavanje logičkih problema. Sa programiranjem se prvi put susreo u sedmom razredu osnovne škole, u periodu kada je počeo da pohađa nastavu u Gimnaziji „Jovan Jovanović Zmaj“. Tada je počeo i da učestvuje na takmičenjima iz programiranja, kojima se bavio tokom narednih godina.



U srednjoj školi izabran je za mentorsku nastavu kod dr Marka Savića. Znanje iz takmičarskog programiranja razvijao je kroz mentorski rad, a zatim i kroz samostalno učenje. Godine 2019. plasirao se na Međunarodnu olimpijadu iz informatike (IOI), na kojoj je osvojio bronzanu medalju.

Po preporuci Marka Savića, školovanje je nastavio na studijskom programu Računarske nauke Prirodno-matematičkog fakulteta Univerziteta u Novom Sadu. Osnovne akademske studije završio je 2022. godine, nakon čega je upisao master akademske studije na istom studijskom programu.

Takmičarskim programiranjem nastavio je da se bavi i tokom studija. U timu sa Tadijom Šebezom i Viktorom Lučićem, pod mentorstvom Marka Savića, dva puta se plasirao na svetsko finale Međunarodnog univerzitetskog takmičenja u programiranju (ICPC). Na drugom nastupu, 2025. godine, tim je osvojio bronzanu medalju.

Od 2022. do 2025. godine radio je kao softverski inženjer u kompaniji Microsoft. Godine 2025. prešao je u kompaniju Databricks, u kojoj radi i danas.

U slobodno vreme voli da rešava slagalice i logičke zagonetke, igra video-igre, putuje i provodi vreme sa porodicom i prijateljima.

- [1] Daniel Meister, Shinji Ogaki, Carsten Benthin, Michael J. Doyle, Michael Guthe, Jiří Bittner, „A Survey on Bounding Volume Hierarchies for Ray Tracing“, Computer Graphics Forum, vol. 40, br. 2, str. 683–712, 2021, doi: 10.1111/cgf.142662.
- [2] J. David MacDonald, Kellogg S. Booth, „Heuristics for Ray Tracing Using Space Subdivision“, The Visual Computer, vol. 6, br. 3, str. 153–166, 1990, doi: 10.1007/BF01911006.
- [3] Ingo Wald, „On Fast Construction of SAH-based Bounding Volume Hierarchies“, 2007 IEEE Symposium on Interactive Ray Tracing, str. 33–40, 2007, doi: 10.1109/RT.2007.4342588.
- [4] James T. Kajiya, „The Rendering Equation“, ACM SIGGRAPH Computer Graphics, vol. 20, br. 4, str. 143–150, 1986, doi: 10.1145/15886.15902.
- [5] Jeffrey Goldsmith, John Salmon, „Automatic Creation of Object Hierarchies for Ray Tracing“, IEEE Computer Graphics and Applications, vol. 7, br. 5, str. 14–20, 1987, doi: 10.1109/MCG.1987.276983.
- [6] Amy Williams, Steve Barrus, R. Keith Morley, Peter Shirley, „An Efficient and Robust Ray-Box Intersection Algorithm“, Journal of Graphics Tools, vol. 10, br. 1, str. 49–54, 2005, doi: 10.1080/2151237X.2005.10129188.
- [7] Thiago Ize, „Robust BVH Ray Traversal“, Journal of Computer Graphics Techniques, vol. 2, br. 2, str. 12–27, 2013, <https://jcgt.org/published/0002/02/02/>.
- [8] NVIDIA Corporation, „CUDA C++ Programming Guide, Version 12.6“, NVIDIA Corporation, 2024, <https://docs.nvidia.com/cuda/archive/12.6.0/cuda-c-programming-guide/index.html>.
- [9] Timo Aila, Samuli Laine, „Understanding the Efficiency of Ray Traversal on GPUs“, Proceedings of High Performance Graphics 2009, str. 145–149, 2009, doi: 10.1145/1572769.1572792.
- [10] Timo Aila, Tero Karras, Samuli Laine, „On Quality Metrics of Bounding Volume Hierarchies“, Proceedings of High Performance Graphics 2013, str. 101–107, 2013, doi: 10.1145/2492045.2492056.
- [11] Martin Stich, Heiko Friedrich, Andreas Dietrich, „Spatial Splits in Bounding Volume Hierarchies“, Proceedings of High Performance Graphics 2009, str. 7–13, 2009, doi: 10.1145/1572769.1572771.
- [12] Andrew Kensler, „Tree Rotations for Improving Bounding Volume Hierarchies“, 2008 IEEE Symposium on Interactive Ray Tracing, str. 73–76, 2008, doi: 10.1109/RT.2008.4634624.
- [13] Tero Karras, Timo Aila, „Fast Parallel Construction of High-Quality Bounding Volume Hierarchies“, Proceedings of High Performance Graphics 2013, str. 89–99, 2013, doi: 10.1145/2492045.2492055.
- [14] Daniel Meister, Jiří Bittner, „Parallel Reinsertion for Bounding Volume Hierarchy Optimization“, Computer Graphics Forum, vol. 37, br. 2, str. 463–473, 2018, doi: 10.1111/cgf.13376.
- [15] Jiří Bittner, Vlastimil Havran, „RDH: Ray Distribution Heuristics for Construction of Spatial Data Structures“, Proceedings of the 25th Spring Conference on Computer Graphics, str. 51–58, 2009, doi: 10.1145/1980462.1980475.
- [16] Marek Vinkler, Vlastimil Havran, Jiří Sochor, „Visibility Driven BVH Build Up Algorithm for Ray Tracing“, Computers & Graphics, vol. 36, br. 4, str. 283–296, 2012, doi: 10.1016/j.cag.2012.02.013.
- [17] Tero Karras, „Maximizing Parallelism in the Construction of BVHs, Octrees, and k-d Trees“, High Performance Graphics 2012, str. 33–37, 2012, doi: 10.2312/EGGH/HPG12/033-037.
- [18] Jacopo Pantaleoni, David Luebke, „HLBVH: Hierarchical LBVH Construction for Real-Time Ray Tracing of Dynamic Geometry“, High Performance Graphics 2010, str. 87–95, 2010, doi: 10.2312/EGGH/HPG10/087-095.

- [19] Matt Pharr, Wenzel Jakob, Greg Humphreys, *Physically Based Rendering: From Theory to Implementation*, 4. izd., MIT Press, 2023, <https://pbr-book.org/4ed/>.
- [20] Daniel Meister, Jiří Bittner, „Parallel Locally-Ordered Clustering for Bounding Volume Hierarchy Construction“, *IEEE Transactions on Visualization and Computer Graphics*, vol. 24, br. 3, str. 1345–1353, 2018, doi: 10.1109/TVCG.2017.2669983.
- [21] Carsten Benthin, Radosław Drabinski, Lorenzo Tessari, Addis Dittebrandt, „PLOC++: Parallel Locally-Ordered Clustering for Bounding Volume Hierarchy Construction Revisited“, *Proceedings of the ACM on Computer Graphics and Interactive Techniques*, vol. 5, br. 3, 2022, doi: 10.1145/3543867.
- [22] Henri Ylitie, Tero Karras, Samuli Laine, „Efficient Incoherent Ray Traversal on GPUs Through Compressed Wide BVHs“, *Proceedings of High Performance Graphics 2017*, str. 4:1–4:13, 2017, doi: 10.1145/3105762.3105773.
- [23] Michal Hapala, Tomáš Davidovič, Ingo Wald, Vlastimil Havran, Philipp Slusallek, „Efficient Stack-less BVH Traversal for Ray Tracing“, *Proceedings of the 27th Spring Conference on Computer Graphics*, str. 7–12, 2011, doi: 10.1145/2461217.2461219.
- [24] Nikolaos Vitsas, Ioannis Evangelou, Georgios Papaioannou, Anestis Gkaravelis, „Parallel Transformation of Bounding Volume Hierarchies into Oriented Bounding Box Trees“, *Computer Graphics Forum*, vol. 42, br. 2, str. 245–254, 2023, doi: 10.1111/cgf.14758.
- [25] Martin Káčerik, Jiří Bittner, „SAH-Optimized k-DOP Hierarchies for Ray Tracing“, *Proceedings of the ACM on Computer Graphics and Interactive Techniques*, vol. 7, br. 3, 2024, doi: 10.1145/3675391.
- [26] Martin Káčerik, Jiří Bittner, „UBVH: Unified Bounding Volume and Scene Geometry Representation for Ray Tracing“, *Computer Graphics Forum*, vol. 44, br. 8, 2025, doi: 10.1111/cgf.70216.
- [27] Ingo Wald, Sven Woop, Carsten Benthin, Gregory S. Johnson, Manfred Ernst, „Embree: A Kernel Framework for Efficient CPU Ray Tracing“, *ACM Transactions on Graphics*, vol. 33, br. 4, 2014, doi: 10.1145/2601097.2601199.
- [28] Marko Savić, „GfxLab-2022-2023“, 2023, <https://github.com/marsavic/GfxLab-2022-2023/tree/master>.
- [29] Stefan Zellmann, Ulrich Lang, „C++ Compile Time Polymorphism for Ray Tracing“, *Proceedings of the 22nd International Symposium on Vision, Modeling and Visualization*, str. 129–136, 2017, doi: 10.2312/vmv.20171268.
- [30] Samuli Laine, Tero Karras, Timo Aila, „Megakernels Considered Harmful: Wavefront Path Tracing on GPUs“, *Proceedings of High Performance Graphics 2013*, str. 137–143, 2013, doi: 10.1145/2492045.2492060.
- [31] Daniel Meister, Jiří Bittner, „Performance Comparison of Bounding Volume Hierarchies for GPU Ray Tracing“, *Journal of Computer Graphics Techniques*, vol. 11, br. 4, str. 1–19, 2022, <https://jcgt.org/published/0011/04/01/>.
- [32] Kirill Garanzha, Jacopo Pantaleoni, David K. McAllister, „Simpler and Faster HLBVH with Work Queues“, *Proceedings of High Performance Graphics 2011*, str. 59–64, 2011, doi: 10.1145/2018323.2018333.
- [33] Daniel Kopta, Thiago Ize, Josef Spjut, Erik Brunvand, Al Davis, Andrew Kensler, „Fast, Effective BVH Updates for Animated Scenes“, *Proceedings of the ACM SIGGRAPH Symposium on Interactive 3D Graphics and Games*, str. 197–204, 2012, doi: 10.1145/2159616.2159649.

UNIVERZITET U NOVOM SADU
PRIRODNO-MATEMATIČKI FAKULTET

KLJUČNA DOKUMENTACIJSKA INFORMACIJA

Redni broj:

RBR

Identifikacioni broj:

IBR

Tip dokumentacije:

TD

Monografska dokumentacija

Tip zapisa:

TZ

Tekstualni štampani materijal

Vrsta rada:

VR

Master rad

Autor:

AU

Nikola Pešić

Mentor:

MN

dr Marko Savić

Naslov rada:

NR

BVH konstrukcija ponderisana cenom primitiva za GPU ray tracing analitičkih scena

Jezik publikacije:

JP

srpski (latinica)

Jezik izvoda:

JI

srpski/engleski

Zemlja publikovanja:

ZP

Srbija

Uže geografsko područje:

UGP

Vojvodina

Godina:

GO

2026

Izdavač:

IZ

Autorski reprint

Mesto i adresa:

MA

Prirodno-matematički fakultet, Trg Dositeja Obradovića 4, Novi Sad

Fizički opis rada:

FO

8/60/33/0/8/11/0

Naučna oblast:

NO

Računarske nauke

Naučna disciplina:

ND

Računarska grafika

Predmetna odrednica/ ključne reči:

PO

UDK

praćenje zraka, hijerarhija ograničavajućih zapremina, BVH, heuristika zasnovana na površini, SAH, GPU, CUDA, analitičke primitive, ponderisana konstrukcija, ponovljivost merenja

Čuva se:

ČU

Biblioteka Departmana za matematiku i informatiku, PMF-a u Novom Sadu

Važna napomena:

VN

nema

Izvod:

IZ

Praćenje zraka (engl. ray tracing) za svaki segment svetlosne putanje traži najbliži presek sa scenom. Hijerarhija

ograničavajućih zapremina (engl. Bounding Volume Hierarchy, BVH) ubrzava ovaj upit grupisanjem primitiva u stablo osno poravnatih ograničavajućih kutija (engl. Axis-Aligned Bounding Box, AABB). Heuristika zasnovana na površini (engl. Surface Area Heuristic, SAH) bira raspored prema površinama čvorova i procenjenom radu u listovima. Standardni objektni SAH procenjuje rad u listu samo brojem primitiva, iako testovi preseka sa različitim vrstama primitiva nemaju jednaku cenu na grafičkom procesoru.

Osnovna ideja rada je da cena sadržaja lista pri konstrukciji zavisi od kalibrisanih relativnih cena preseka sa sferom, kutijom, afinom sferom i afinom kutijom. Ispitana su četiri načina konstrukcije opisana u literaturi: objektni SAH, stabla razdvojena po tipu primitive, SBVH i lokalne rotacije, svaki u standardnoj i ponderisanoj varijanti. Metode su poređene vremenom CUDA kernela za ceo render na tri vrste opterećenja. Namenski konstruisana familija Layered Hero ispituje osetljivost i rast pravilne heterogene scene, Overlap proverava prostorne podele pri snažnom preklapanju AABB granica, a familije F1 - F4 proveravaju ponašanje na nasumičnim rasporedima i različitim sastavima primitiva.

Medijana promene vremena GPU kernela ponderisanog objektnog SAH prema standardnom objektnom SAH kroz pet skala familije Layered Hero iznosi 7,56%, pri čemu pozitivan znak označava kraće vreme. Ponderisanje skraćuje medijansko vreme i kod stabala po tipu, SBVH konstrukcije i lokalnih rotacija, uz učinak koji zavisi od porodice i scene.

Datum prihvatanja teme od NN veća:
DP

1. jul 2026.

Datum odbrane:
DO

Članovi komisije:
KO

Predsednik:

dr Miloš Stojaković, redovni prof. Prirodno-matematičkog fakulteta u Novom Sadu

Član:

dr Miloš Racković, redovni prof. Prirodno-matematičkog fakulteta u Novom Sadu

Član, mentor:

dr Marko Savić, docent Prirodno-matematičkog fakulteta u Novom Sadu

UNIVERSITY OF NOVI SAD
FACULTY OF SCIENCE AND MATHEMATICS

KEY WORDS DOCUMENTATION

Accession number:

ANO

Identification number:

INO

Document type:

DT

Monograph publication

Type of record:

TR

Textual printed material

Content code:

CC

Master thesis

Author:

AU

Nikola Pešić

Mentor/comentor:

MN

dr Marko Savić

Title:

TI

Primitive-Cost-Weighted BVH Construction for GPU Ray Tracing of Analytic Scenes

Language of text:

LT

Serbian (Latin)

Language of abstract:

LA

Serbian/English

Country of publication:

CP

Serbia

Locality of publication:

LP

Vojvodina

Publication year:

PY

2026

Publisher:

PU

Author's reprint

Publication place:

PP

Faculty of Science, Trg Dositeja Obradovića 4, Novi Sad

Physical description:

PD

8/60/33/0/8/11/0

Scientific field:

SF

Computer Science

Scientific discipline:

SD

Computer Graphics

Subject/ Key words:

SKW

UC

ray tracing, Bounding Volume Hierarchy, BVH, Surface Area Heuristic, SAH, GPU, CUDA, analytic primitives, weighted construction, measurement reproducibility

Holding data:

HD

Library of Department of Mathematics and Informatics, Trg Dositeja Obradovića 4

Note:

N

none

Abstract:

AB

Ray tracing requires a closest-hit query against the scene for every light-path segment. A Bounding Volume Hierarchy (BVH)

accelerates this query by grouping primitives in a tree of Axis-Aligned Bounding Boxes (AABBs). The Surface Area Heuristic (SAH) chooses the arrangement from node surface areas and estimated leaf work. Standard object SAH estimates leaf work from the primitive count even though their intersection tests do not have equal costs on the GPU.

This thesis implements eight methods, organized as standard and weighted pairs for object SAH, trees separated by primitive type, SBVH, and local rotations. The weighted variants use calibrated relative costs for spheres, boxes, affine spheres, and affine boxes. The methods are compared using whole-render CUDA-kernel time on three kinds of workloads. The purpose-built Layered Hero family tests sensitivity and scaling in a regular heterogeneous scene, Overlap tests spatial splits under strong AABB overlap, and the F1 - F4 families test random layouts and different primitive compositions.

Across the five Layered Hero scales, the median GPU-kernel time change of weighted object SAH relative to standard object SAH is 7.56%, where a positive sign denotes a shorter time. Weighting also reduces median time for trees separated by type, SBVH, and local rotations, with an effect that depends on the family and scene.

Accepted by the Scientific Board:
ASB

1 July 2026

Defended on:
DE

Thesis defence board:
DB

President:

Dr Miloš Stojaković, Full Professor, Faculty of Sciences,
University of Novi Sad

Member:

Dr Miloš Racković, Full Professor, Faculty of Sciences,
University of Novi Sad

Member, mentor:

Dr Marko Savić, Assistant Professor, Faculty of Sciences,
University of Novi Sad